ECOLE POLYTECHNIQUE DE BRUXELLES

INFO-H419

Université Libre de Bruxelles

École Polytechnique de Bruxelles

Data Warehouses

Decision Support Benchmark (TPC-DS)
Oracle Database

Authors:

DIALLO Thierno
GÓMEZ MARTÍN Martina
BÂAJ Amine
SHAROF Imad

Professor:

ZIMANYI Esteban

November 8th, 2026

Contents

1 Introduction **5**

2 Oracle Setup and Experimental Limitations **5**
2.1 Hardware and Storage Limitations 5
2.1.1 Experimental Environement 6
2.2 Optimizer Statistics 6
2.3 Benchmark Adaptations 7

3 Data and Query Preparation **7**
3.1 TPC-DS Benchmarking Set-Up 7
3.1.1 Data Generation 7
3.1.2 Schema Set-Up 8
3.1.3 Data Load 8
3.1.4 Query Generation 8
3.2 Query Adaptation for Oracle 8
3.2.1 Date Literal Compatibility 9
3.2.2 Simplification of Date Casts 9
3.2.3 Except Statement Issue 9
3.2.4 Query Structure Adjustments 10
3.2.5 Schema Column Name Correction 10
3.3 Validations 10
3.3.1 Data Validation 10
3.3.2 Query Validation 10

4 Benchmark Implementation **11**
4.1 Benchmarking 11
4.1.1 Power Test (Sequential Test) 11
4.1.2 Throughput Test 1 and 2 11
4.1.3 Data Maintenance 11
4.1.3.1 Execution of a refresh run 12
4.1.3.2 Validation 13
4.1.3.3 Oracle adaptations 13
4.1.3.4 Results 13
4.1.4 Performance Metric 14

5 Results and Analysis **15**
5.1 Scaling 15
5.2 Elapsed Time of Each Test 15
5.2.0.1 Load Test 16
5.3 Distribution of the Query Times 17

5.4 Slowest Queries . . . 17
5.5 Query Profiles . . . 18
5.5.0.1 Linear queries. . . . 19
5.5.0.2 Superlinear queries. . . . 19
5.6 Query 78 . . . 20
5.7 Throughput Tests against the Power Test . . . 20
5.8 Data Maintenance . . . 21
5.9 QphDS across Scale Factors . . . 22
5.10 Limitations . . . 23

6 Query Optimization 23
6.1 Overview . . . 24
6.2 Gathering statistics . . . 24
6.3 Primary and foreign key relationships . . . 25
6.4 Optimizer configuration . . . 25
6.4.1 Why we tested these changes. . . . 25
6.5 Indexes . . . 25
6.5.0.1 Why we expected an improvement. . . . 25
6.5.0.2 Why there was no improvement. . . . 26
6.6 Materialized views . . . 26
6.6.1 How the views were designed . . . 26
6.6.2 Summary . . . 26
6.6.3 Views created and intended reuse . . . 27
6.6.4 Why the total may still be worse . . . 27
6.7 Limitations . . . 27

7 Query-level Optimization 28
7.1 Query 78 . . . 28
7.1.1 Query Structure . . . 30
7.1.2 Execution Plan Analysis . . . 31
7.1.3 Optimization Experiments . . . 32
7.1.3.1 SQL Rewriting with UNION ALL . . . 32
7.1.3.2 Optimization with Oracle Hints . . . 33
7.1.4 Results and Discussion . . . 33
7.2 Query 72 . . . 34
7.2.1 Query Structure . . . 34
7.2.2 Reading the execution plan . . . 35
7.2.3 Execution Plan Analysis . . . 36
7.2.4 Optimization Experiments . . . 37
7.2.4.1 Moving conditions to the `ON` clauses. . . . 37
7.2.4.2 SQL rewriting with a CTE. . . . 37
7.2.4.3 Optimizer hints. . . . 38
7.2.5 Results and Discussion . . . 39

7.3 Query 63 40

8 Conclusion **40**

9 Appendix **42**

1 Introduction

This report presents the first part of the Data Warehouses (INFO-H419) project, focusing on an adapted version of the TPC-DS benchmark on Oracle Database. The main objective is to study the impact of different dataset sizes (scale factors) on the execution times of the same set of queries. To investigate this, We generated and loaded datasets at five scale factors, executed the 99 analytical queries and measured their performance in sequential and concurrent tests. We also studied several query optimization techniques using Oracle execution plans.

This project is an adapted implementation rather than a fully compliant TPC-DS benchmark. The main adaptations and limitations are described throughout the report.

2 Oracle Setup and Experimental Limitations

For this project,we used Oracle Database Free to implement an adapted version of the TPC-DS benchmark. Although it provides the main features needed for our benchmark, this edition has several resource limitations that influenced our experimental setup.

2.1 Hardware and Storage Limitations

Although the benchmark was executed on different machines (the final benchmark measurements were performed on a single machine), Oracle Database Free imposes the same resource limits regardless of the hardware available. These restrictions, summarized in Table 1, can affect performance when executing complex analytical queries or working with larger datasets.

Table 1: Oracle Database Free resource limitations

Resource	Limit
CPU cores	2
RAM	2 GB
User data storage	12 GB

The storage limit was particularly important when choosing our dataset sizes. We used the dsdgen tool provided by the TPC-DS toolkit to generate data at different scale , with scale 1 representing approximately 1 GB of raw data. However, the actual space occupied in Oracle is larger due to additional structures such as indexes. For example, our scale 1 dataset occupied approximately 1.84 GB in the database. As the scale increases, this additional storage becomes a significant constraint. We therefore limited our experiments to scale 1, 2, 3, 5 and 7, with scale SF7 being the largest dataset tested and our reference scale factor (Following the project guidelines, that correspond to the relative scale factor 1.0). The

other scales factors correspond to 1/7, 2/7, 3/7 and 5/7 of this reference.For consistency, throughout the report, we use the notation SF1, SF2, SF3, SF5, and SF7 to refer to the actual scale factors used by the data generator and benchmark scripts.

The data generation and loading processes are described in the following sections.

2.1.1 Experimental Environement

All benchmark measurements were performed on the same machine, with the following configuration:

Component	Configuration
Machine	Apple MacBook Pro M4
Database	Oracle Database Free
Execution environment	Docker container

Oracle was executed inside a Linux container running through a virtual machine, which may also affect disk I/O. Despite the machine's available resources, the database remains restricted by the Oracle Database Free edition limitations as described above.

2.2 Optimizer Statistics

Table 2: Slowest queries during the initial Power Test ($SF5$)

Position	Query ID	Elapsed Time (s)
47	35	Still running after 3 hours
28	69	3362.43
26	10	572.11
41	17	127.84
10	78	56.52
8	19	51.16
5	39	26.73
2	7	25.82
35	67	25.14
33	85	17.31
36	28	13.85

Position refers to the relative execution order of each query in the Stream 0

Another important aspect we encountered was the role of optimizer statistics in query performance. Oracle uses these statistics to estimate the cost of different operations and select an appropriate execution plan. During an initial Power Test with SF5, we had not explicitly collected table statistics after loading the data, which led Oracle to rely on dynamic

statistics for some queries.We noticed that several queries were taking considerably longer than expected, as shown in Table 2. In particular, Query 35 was still running after more than three hours and had to be stopped, while Query 69 took approximately 56 minutes to complete.

After examining the execution plans, we suspected that the lack of collected statistics could have contributed to inefficient execution strategies. To address this issue, we included the `DBMS_STATS.GATHER_SCHEMA_STATS(USER)` procedure at the end of the data loading process, ensuring that Oracle has updated statistics before executing the benchmark queries (the time required for statistics collection is included in the Load Test).

2.3 Benchmark Adaptations

We made several adjustments to the official TPC-DS benchmark to simplify the experiments and make easy the comparison of the results. The main adaptations are:

- **Fixed query parameters:** For each query, we keep the same substitution parameters across all scale factors allowing to compare execution times more consistently and better isolate the impact of increasing dataset size without introducing variations caused by different query parameters.
- **Query stream execution:** During the Throughput Tests, we follow the predefined stream permutations to change the execution order while keeping the query parameters unchanged.

3 Data and Query Preparation

3.1 TPC-DS Benchmarking Set-Up

3.1.1 Data Generation

The TPC-DS benchmark is used to evaluate the performance of Oracle under a decision-support workload. The benchmark provides a standardised set of database tables, generated datasets, and SQL queries designed to represent analytical workloads.

The dataset is generated using the TPC-DS data-generation tools (dsdgen), following the workflow described in the benchmark documentation.

The generated data is stored in `.dat` files, each containing the records corresponding to a table in the TPC-DS schema. These files are then used to populate the Oracle database, as described in the following section.

3.1.2 Schema Set-Up

After generating the dataset, the database schema is created in Oracle using the tpcds.sql script provided with the project setup. This script defines the tables required by the benchmark and establishes the database structure into which the generated data is loaded. The table definitions provided the reference for the data-loading process, including column names, data types, and column order.

3.1.3 Data Load

The generated `.dat` files are loaded into Oracle using a custom Python script. This approach is chosen to provide a consistent and reproducible data-loading procedure across the different development environments used by the project team.

The script uses the `python-oracledb` library to connect to the database and load the generated data files into their corresponding tables. This automated approach simplifies the loading process and ensures that the same procedure can be used across the different environments.

3.1.4 Query Generation

The TPC-DS benchmark provides a set of SQL query templates that represent analytical workloads for decision-support systems. According to the TPC-DS documentation, the `dsqgen` utility is used to transform these templates into executable SQL queries. The generated queries incorporate parameter values and are adapted to the SQL dialect of the target database system. In this project, the Oracle dialect was selected to generate queries for execution in Oracle Database.

After generation, the SQL queries were executed against the Oracle database populated with the TPC-DS dataset. Some generated queries required additional modifications to ensure compatibility with the Oracle environment. These modifications are described in the following section.

3.2 Query Adaptation for Oracle

The SQL queries generated by the TPC-DS query-generation scripts required several modifications to execute successfully in the Oracle environment. These modifications primarily involved adapting date expressions to Oracle's SQL syntax, correcting column references that did not match the schema, and restructuring certain queries that contained multiple concatenated SQL statements. The following sections describe the main compatibility issues encountered, the corresponding changes implemented, and examples illustrating how the generated queries were adapted for execution.

3.2.1 Date Literal Compatibility

Oracle's ANSI date literal syntax explicitly identifies the value as a date in the YYYY-MM-DD format. It also avoids relying on implicit string-to-date conversion and the session's date format settings. Therefore these expressions were modified to use Oracle's date literal syntax.

For example, the following expression:

```
CAST('1998-08-04' AS DATE)
```

was replaced with:

```
DATE '1998-08-04'
```

A similar modification was required for date arithmetic. An expression such as:

```
CAST('1998-08-04' AS DATE) + 14 days
```

was replaced with:

```
DATE '1998-08-04' + 14
```

Comparisons between date columns and string literals were also updated. For example:

```
d_date = '1998-02-19'
```

was changed to:

```
d_date = DATE '1998-02-19'
```

3.2.2 Simplification of Date Casts

Some generated queries contained expressions that explicitly cast date columns to the DATE type, for example:

```
CAST(d_date AS DATE)
```

These expressions were simplified to:

```
d_date
```

This modification was applied where the underlying column was already defined as an Oracle DATE. In such cases, the cast is redundant because the column already has the required data type.

The change was made without altering the underlying column definition.

3.2.3 Except Statement Issue

In Query 87, the structure expresses the intention to calculate the difference between the results of multiple queries. However, the parentheses around individual SELECT statements causes syntax compatibility issues in SQL dialects that do not accept this form of set operation.

To preserve EXCEPT, the individual queries were wrapped in nested subqueries instead of placing parentheses directly around each SELECT.

3.2.4 Query Structure Adjustments

Queries 14, 23, 24, and 39 required additional modifications because their generated SQL files contained two concatenated query statements. This structure prevented the statements from being executed as intended in the Oracle environment. To address this issue, each file was divided into two separate subqueries, allowing the statements to be executed independently. This modification concerned the organisation of the generated SQL files rather than the underlying query logic. The resulting subqueries were then used for subsequent execution and testing.

3.2.5 Schema Column Name Correction

An additional compatibility issue was identified in Query 30. The query referenced the column c_last_review_date_sk, which did not exist in the schema created for the project.

The reference was corrected to c_last_review_date, matching the column defined in the actual schema. This resolved the invalid column reference and allowed the query to use the appropriate customer review-date attribute.

3.3 Validations

After generating the data and queries with the TPC-DS toolkit and adapting them to Oracle, two validation steps were performed to verify the correctness of the loaded data and the results returned by the adapted queries.

3.3.1 Data Validation

The data loading process was validated through three checks:

- **Row counts:** The number of records in the generated `.dat` files was compared with the number of rows loaded into the corresponding Oracle tables.
- **Data sampling:** The TPC-DS `dsdgen` validation tool was used to generate at least 50 reference records per table. These records were then compared with their corresponding values in the Oracle tables.
- **Character encoding:** A few records containing international characters, such as accented letters, were manually checked to verify their correct storage.

3.3.2 Query Validation

The adapted queries were validated using the TPC-DS reference answer sets available for SF1 with predefined query parameters. Their results were compared with the expected outputs to verify that the Oracle-specific syntax modifications did not affect the correctness of the

queries. These modifications were therefore kept and applied to the benchmark queries with their actual parameters.

4 Benchmark Implementation

4.1 Benchmarking

Both the Power Test and the Throughput Tests use the same python script (**benchmark_queries.py**), to execute and measure SQL queries. For each stream, the script loads the 99 queries in the order defined by **streams_order.json**, opens a connection to Oracle and executes queries sequentially.

4.1.1 Power Test (Sequential Test)

For the Power Test, the Bash script (**tpcds.sh**) launches a single instance of **benchmark_queries.py** with stream 0. All 99 queries are executed sequentially within one Oracle session. The total execution time (T_{Power}) is measured from the beginning of the first query to the completion of the last one including the time needed to retrieve the query results.

4.1.2 Throughput Test 1 and 2

The throughput test follows the same execution procedure but uses 4 concurrent streams instead of one: four independent python processes are lunched in the background by the bash script, each with its own Oracle connection and query execution order (located in the **streams_order.json**).

- **Throughput Test 1:** uses the relative order of the queries provided in streams 1 -> 4.
- **Throughput Test 2:** uses the relative order of the queries provided in streams 5 -> 8.

The 99 queries are still executed sequentially but the four stream run concurrently. The total duration (T_{TT1} and T_{TT2}) are calculated from the earliest query start time across all the four streams to the latest query completion time.

4.1.3 Data Maintenance

A data warehouse is not static, it is periodically refreshed with new data coming from the operational (OLTP) systems. TPC-DS models this with the Data Maintenance (DM) test (Clause 5 of the specification). It is executed twice during the benchmark, after each Throughput Test:

- **DM1** runs refresh runs 1 and 2, after Throughput Test 1;
- **DM2** runs refresh runs 3 and 4, after Throughput Test 2.

With $S_q = 4$ query streams, the specification requires $S_q/2 = 2$ refresh runs per DM test, so 4 refresh data sets are needed in total. They are generated with `dsdgen -update` n ($n = 1 \ldots 4$), which writes, for each refresh data set, the new business transactions as flat files (`s_purchase_`n`.dat`, `s_catalog_order_`n`.dat`, `s_web_order_`n`.dat`, their line items, returns and inventory) and the date ranges of the data to remove (`delete_`n`.dat` and `inventory_delete_`n`.dat`). Data generation is not timed.

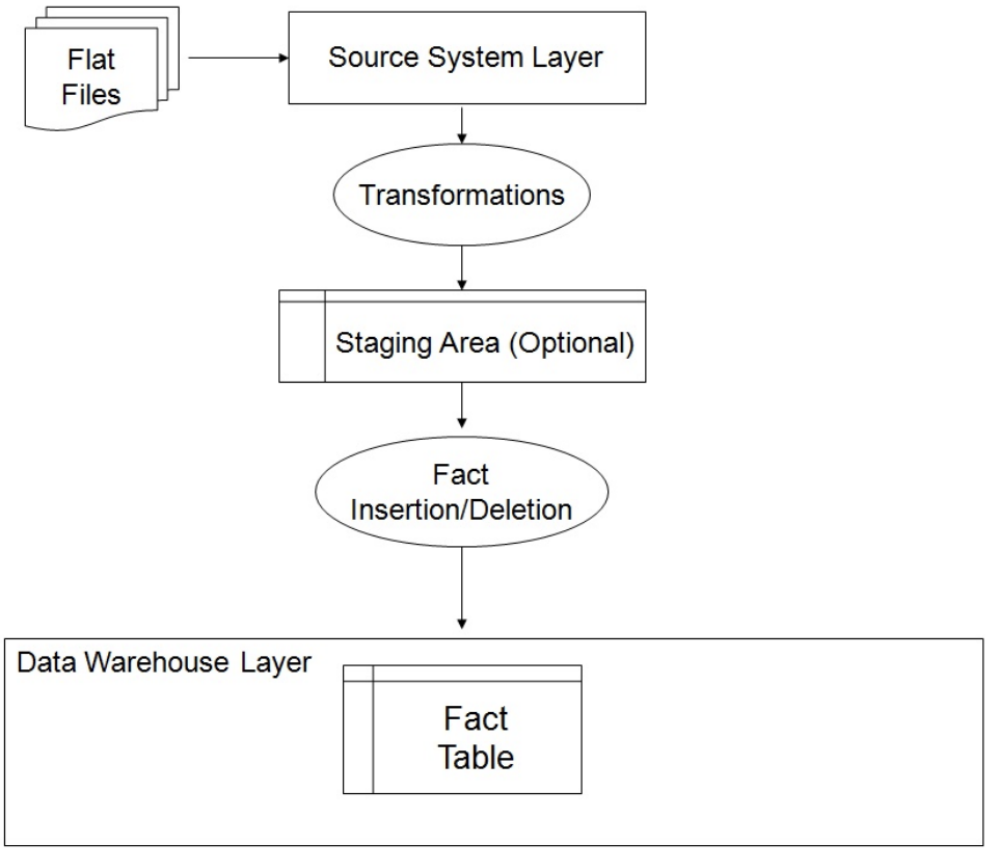


Figure 1: Execution Overview of the Data Maintenance Process

4.1.3.1 Execution of a refresh run

in `data_maintenance.py`. Each refresh run follows four main steps:

1. **Load:** Load the generated data into staging tables.
2. **Delete:** Remove the specified sales, returns and inventory records.
3. **Insert:** Insert the new records into the warehouse tables using SQL views that transform the staging data.
4. **Commit:** Save the changes to the database.

The execution time is measured from the start of the first refresh run to the end of the second, including the staging data load.

4.1.3.2 Validation

We verified the Data Maintenance process by comparing the inserted row counts with the expected values. For example at SF1, the first refresh run inserted 293 589 rows, matching the expected total.

4.1.3.3 Oracle adaptations

The views of the specification were adapted to Oracle and to the data produced by dsdgen 4.0.0, with minor modifications:

- `CAST(x AS DATE)` was replaced by `TO_DATE(x, 'YYYY-MM-DD')`, since Oracle's default date format is `DD-MON-RR`.
- dsdgen restarts the ticket and order numbers at 1 in every refresh data set, which violates the primary keys of the fact tables. Before the inserts, the current maximum number of each channel is stored in a table `dm_offset` and added to the new numbers.
- In `s_catalog_order_lineitem`, dsdgen writes the surrogate key of the catalog page instead of its page number, so `csv` joins `catalog_page` on `cp_catalog_page_sk`.
- At scale factor 2, dsdgen generates an item whose only version is closed (`i_rec_end_date` not null). The condition `i_rec_end_date IS NULL` then finds no item, and the insert fails on the `NOT NULL` column `ws_item_sk`. The views `wsv`, `csv` and `iv` therefore join the latest version of each item:

```
CREATE OR REPLACE VIEW item_latest AS
SELECT * FROM item
WHERE i_item_sk IN (SELECT MAX(i_item_sk) FROM item GROUP BY
   i_item_id);
```

  For every other item, the latest version is the open one, so the results are unchanged.
- After the data maintenance, some item groups have as many returns as sales. Query 75 then divides by zero (`ORA-01476`). Its divisor was wrapped in `NULLIF(..., 0)`.

4.1.3.4 Results

Table 3 shows the time of each phase per refresh run.

This gives $T_{DM1} = 12.7\,\text{s}$ and $T_{DM2} = 12.2\,\text{s}$ at SF1, and $T_{DM1} = 23.4\,\text{s}$ and $T_{DM2} = 25.0\,\text{s}$ at SF2. Three observations:

- The deletes take about 60% of each refresh run. Removing the returns and sales of a date range requires scanning the large fact tables, whose size grows with the scale factor.

SF	Run	Load (s)	Deletes (s)	Inserts (s)	Total (s)	Rows inserted
1	1	0.42	4.32	2.14	6.89	293 589
1	2	0.47	3.23	2.13	5.85	293 604
1	3	0.37	3.55	2.22	6.15	293 604
1	4	0.37	3.61	2.02	6.01	293 557
2	1	0.61	7.77	3.99	12.39	434 999
2	2	0.70	6.68	3.59	10.98	434 994
2	3	0.59	7.87	4.35	12.82	435 041
2	4	0.67	7.77	3.78	12.22	435 015

Table 3: Time per phase of each refresh run

- The inventory accounts for most of the inserted rows (270 000 of 293 589 at SF1), so LF_I is the largest insert function in terms of rows.
- The load of the staging tables and the commit are negligible (under 1 s).

The data maintenance time roughly doubles from SF1 to SF2, which is consistent with the deletes depending on the size of the fact tables. The data maintenance remains small compared with the query tests: about 25 s for both DM tests at SF1, against about 133 s for the two Throughput Tests.

4.1.4 Performance Metric

The primary performance metric of TPC-DS is the number of queries per hour at a given scale factor, QphDS@SF (Clause 7.6.3). It combines the elapsed times of all the tests of the benchmark into a single value:

$$\text{QphDS@SF} = \left\lfloor \frac{SF \cdot Q}{\sqrt[4]{T_{PT} \cdot T_{TT} \cdot T_{DM} \cdot T_{LD}}} \right\rfloor \tag{1}$$

where:

- SF is the scale factor;
- $Q = S_q \times 99$ is the total number of weighted queries, with $S_q = 4$ query streams, i.e. $Q = 396$;
- $T_{PT} = T_{Power} \times S_q$ is the Power Test time, multiplied by the number of streams;
- $T_{TT} = T_{TT1} + T_{TT2}$ is the total time of the two Throughput Tests;
- $T_{DM} = T_{DM1} + T_{DM2}$ is the total time of the two Data Maintenance tests;
- $T_{LD} = 0.01 \times S_q \times T_{Load}$ is the load time, weighted by $0.01 \times S_q$.

All times are expressed in hours.

5 Results and Analysis

5.1 Scaling

We ran the full benchmark at scale factors 1, 2, 3, 5 and 7, with $S_q = 4$ streams. Each run follows the order of the specification: load, Power Test, Throughput Test 1, Data Maintenance 1, Throughput Test 2 and Data Maintenance 2. Before each scale factor, all tables and views of the TPCDS schema are dropped and created again, so every run starts from an empty schema.

SF7 is the largest scale factor that fits in Oracle Database Free. In Oracle, SF5 takes 7.6 GB (5.3 GB of tables and 2.3 GB of primary key indexes), so SF8 would need about 12.2 GB. That is above the 12 GB limit of the free edition. The limits of the free edition explain most of the results in this section (Table 4).

Parameter	Value
CPU threads (`cpu_count`)	2
SGA (`sga_target`)	1.5 GB
of which buffer cache	906 MB
PGA target (`pga_aggregate_target`)	512 MB
Maximum user data	12 GB

Table 4: Resources of the Oracle Database Free instance

The schema only has the primary keys of the TPC-DS tables. We did not add any secondary index.

5.2 Elapsed Time of Each Test

Table 5 and Figure 2 give the elapsed time of each test.

SF	T_{Load}	T_{Power}	T_{TT1}	T_{DM1}	T_{TT2}	T_{DM2}	QphDS@SF
1	124.4	31.1	76.4	12.7	57.0	12.2	37 641
2	263.5	56.9	132.2	23.4	138.1	25.0	38 089
3	440.1	1 254.3	2 951.9	40.0	314.1	55.7	10 492
5	927.6	343.9	883.0	87.1	767.7	103.7	20 019
7	1 471.3	13 558.1	14 402.8	195.7	12 749.7	107.1	4 409

Table 5: Elapsed times (seconds) and QphDS@SF

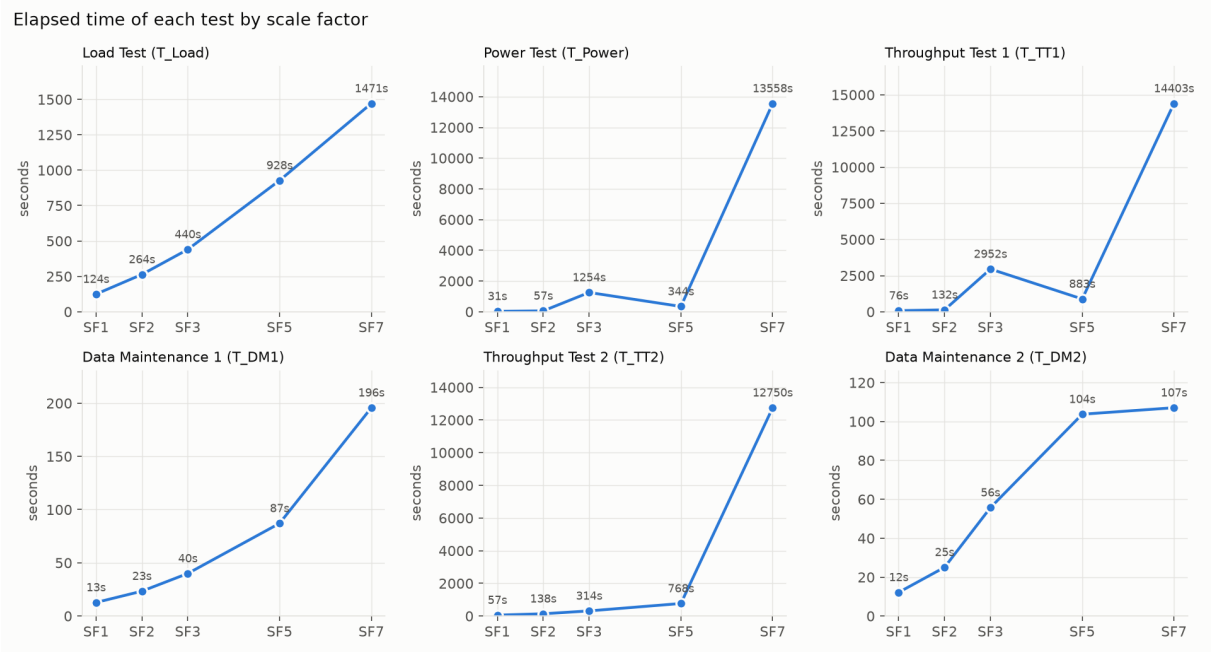


Figure 2: Elapsed time of each test by scale factor

The load and the two data maintenance tests grow steadily, a little faster than the data. The three query tests do not. The Power Test and Throughput Test 1 jump at SF3, go back down at SF5 and explode at SF7, where each query test takes between 3.5 and 4 hours. Throughput Test 2 has no peak at SF3, but it explodes at SF7 too. As shown below, these jumps come almost entirely from one query, Query 78. Without it, the query tests also grow steadily.

5.2.0.1 Load Test

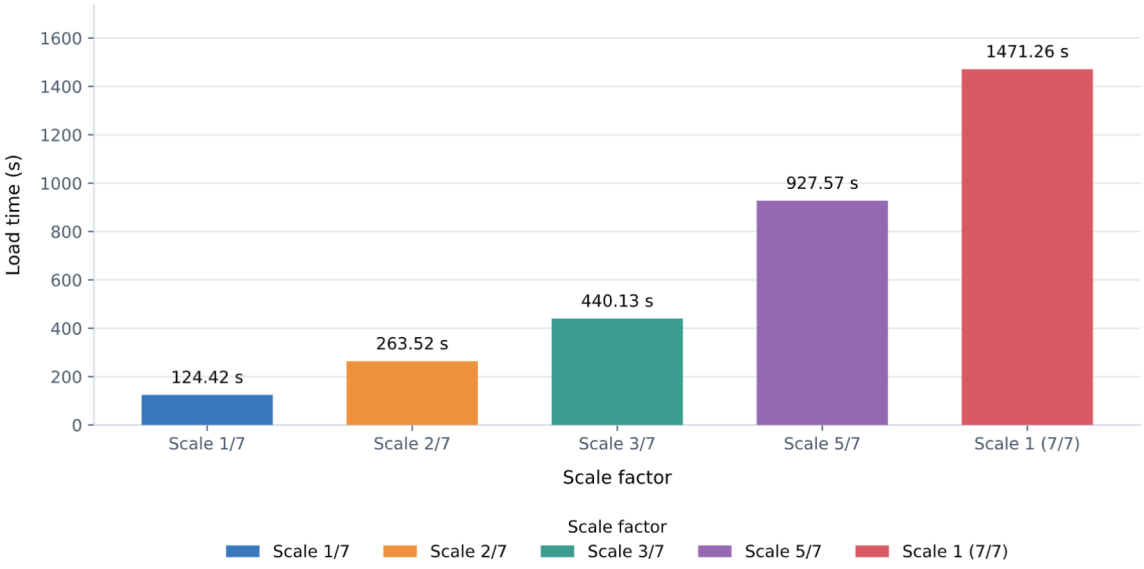


Figure 3: TPC-DS: loading time by scale factor

The load time goes from 124 s at SF1 to 1 471 s at SF7, so it is 11.8 times longer for 7 times more data. It grows in proportion to the data up to SF2 and faster after that. First, the primary key indexes get deeper as the tables grow, so they cost more to maintain. Second, T_{Load} includes gathering the statistics, which reads every table again. At SF7 the `inventory` table alone has 77 million rows, which is 65% of all the loaded rows.

5.3 Distribution of the Query Times

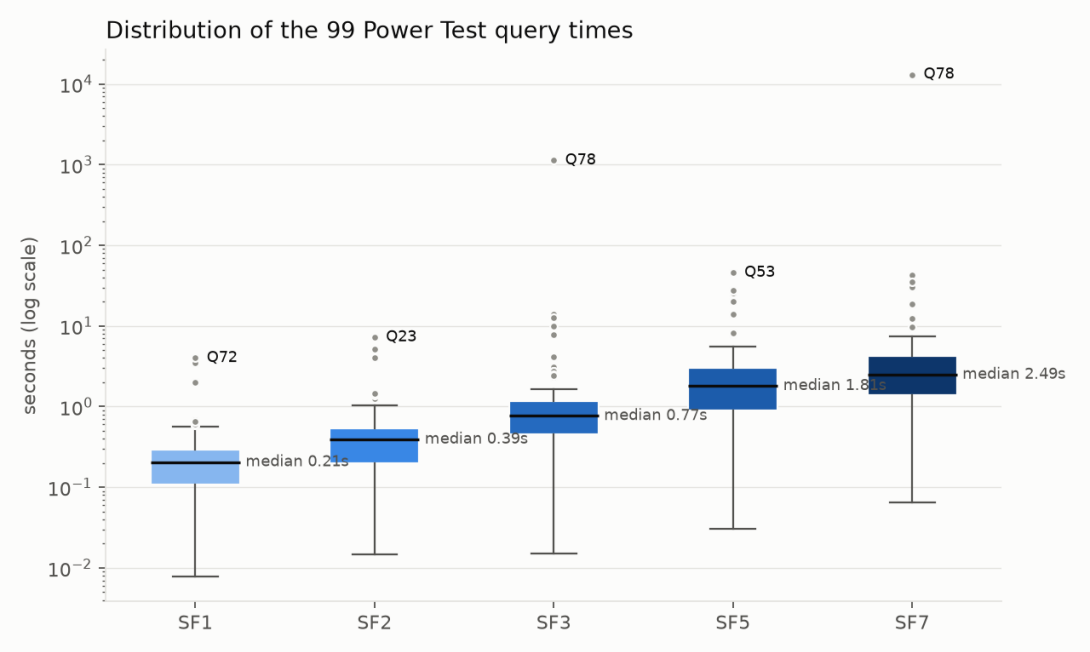


Figure 4: Distribution of the 99 Power Test query times (log scale)

Most queries are short. The median query takes 0.21 s at SF1 and 2.49 s at SF7, so it is 11.9 times slower for 7 times more data. The whole box moves up regularly with the scale factor. The typical query therefore scales a little worse than the data, but without any break.

Each scale factor has a few outliers: Query 72 at SF1, Query 23 at SF2 and Query 53 at SF5. These are heavy queries, but they all stay under one minute. Query 78 at SF3 and SF7 is a different case. It takes 1 127 s and 13 099 s, about a thousand times more than the median query, while at SF5 it takes only 5.6 s and is inside the normal range.

5.4 Slowest Queries

Figure 5 shows the 15 slowest queries of the Power Test at SF7, with their time at every scale factor.

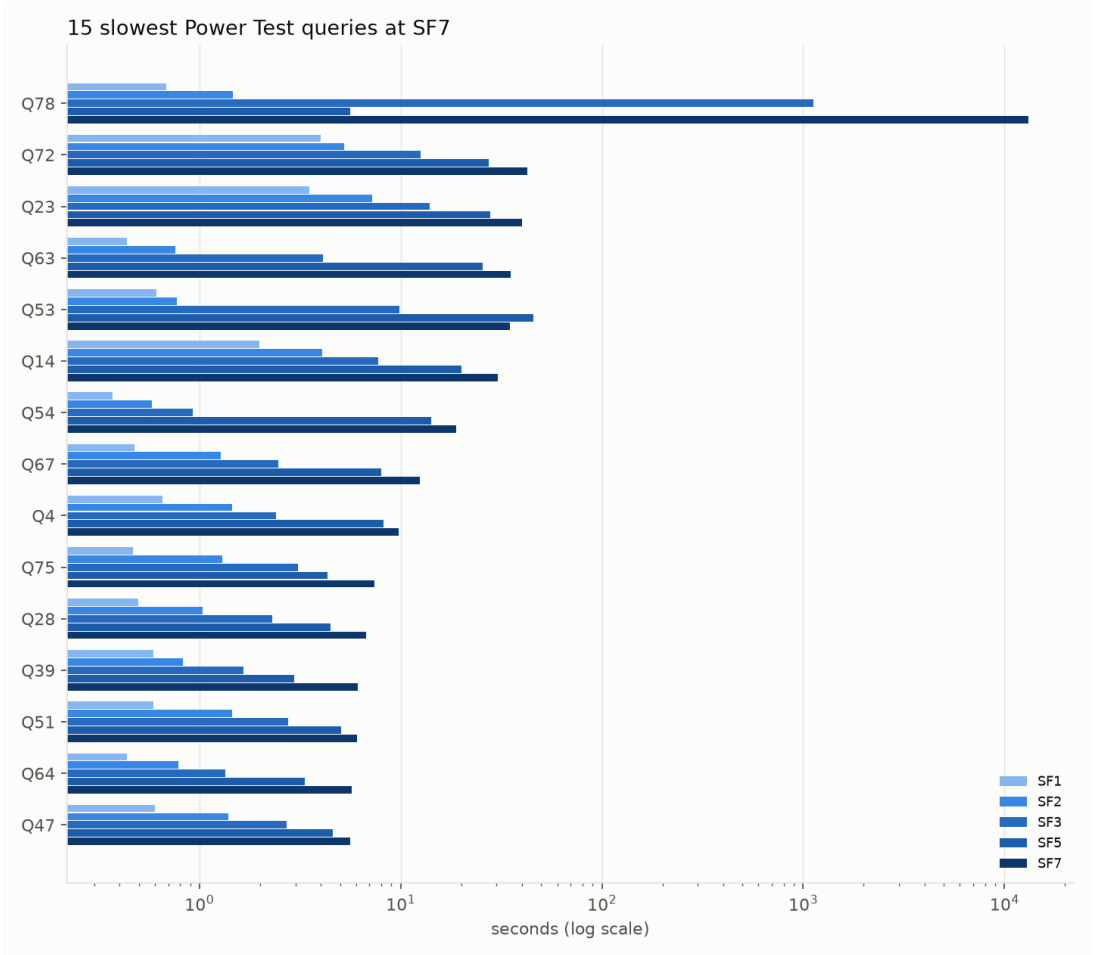


Figure 5: The 15 slowest Power Test queries at SF7, at each scale factor (log scale)

Apart from Query 78, the slowest queries at SF7 include Q72, Q63 and Q23. Some queries are consistently expensive because they process large amounts of sales and inventory data. Others show stronger increases at certain scale factors. Query 78 is particularly unusual because its execution time varies considerably between dataset sizes. This behaviour is analyzed separately in Section 5.6.

5.5 Query Profiles

To classify the queries, we divided the Power Test time of each query at SF7 by its time at SF1. The data is 7 times larger. A query is *linear* if it is at most 14 times slower (twice the data ratio) and *superlinear* if it is slower than that. We left out queries whose time drops by more than a factor of 10 from one scale factor to the next, because their time depends on the plan and not on the data size. Only Query 78 is in this case. With this rule, 54 queries are linear and 44 are superlinear. Figure 6 shows three typical queries of each profile.

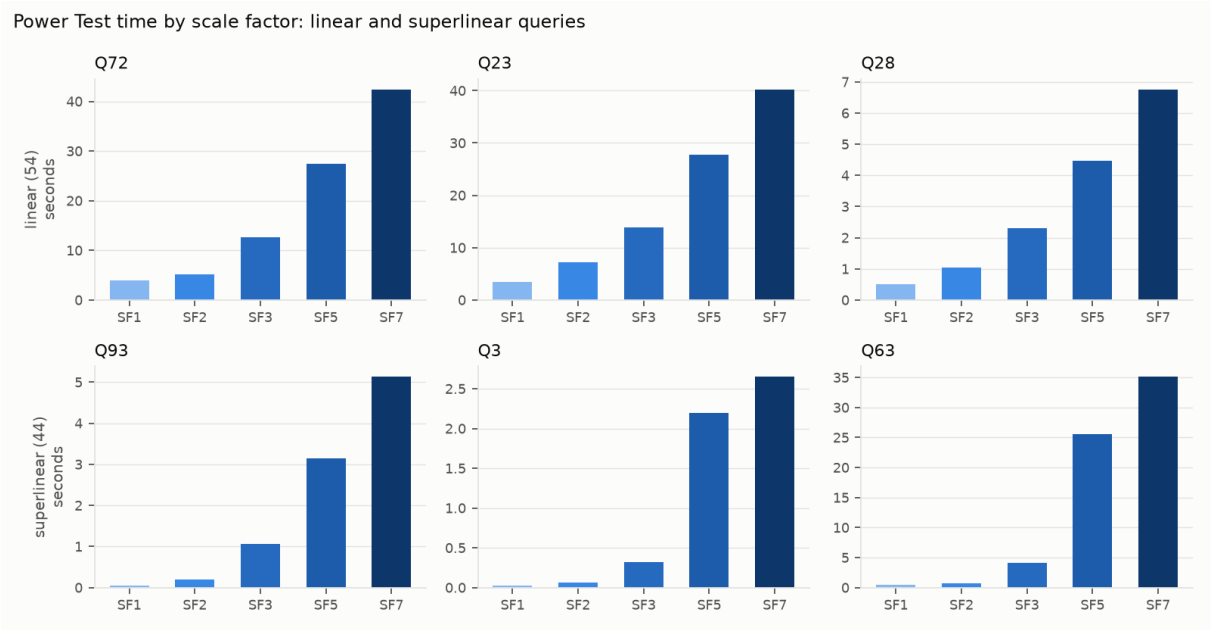


Figure 6: Power Test time by scale factor for typical linear and superlinear queries

5.5.0.1 Linear queries.

Queries 72, 23 and 28 grow in proportion to the data. Query 72 goes from 4.0 s at SF1 to 42.4 s at SF7, Query 23 from 3.5 s to 40.3 s and Query 28 from 0.5 s to 6.8 s. They are among the slowest queries, but their time comes from large scans and joins, and that cost grows with the size of the tables. Being slow does not make a query superlinear: the heaviest queries of the benchmark are all linear.

5.5.0.2 Superlinear queries.

Queries 93, 3 and 63 are very fast at SF1 and SF2 and much slower from SF3 on. Query 3 goes from 0.06 s at SF2 to 2.2 s at SF5, and Query 63 from 0.76 s to 25.6 s. These queries mostly read `store_sales`, the largest fact table. The jump matches the size of the buffer cache. The `store_sales` flat file is 0.78 GB at SF2 and 1.18 GB at SF3, and the buffer cache is 906 MB. Up to SF2 the table stays in memory between queries, so they only read from memory. From SF3 the table no longer fits, and each query has to read it from disk again. Between SF5 and SF7, when the table is read from disk in both cases, these queries grow in proportion to the data again (Query 3: 2.2 s to 2.7 s).

The whole Power Test shows the same effect once Query 78 is removed (Table 6). The time per scale factor unit is about 30 s at SF1 and SF2. It then rises, and it is stable again at about 66 s between SF5 and SF7.

SF	T_{Power} (s)	T_{Power} without Q78 (s)	per SF unit (s)
1	31.1	30.4	30.4
2	56.9	55.4	27.7
3	1 254.3	126.8	42.3
5	343.9	338.2	67.6
7	13 558.1	458.8	65.5

Table 6: Power Test time with and without Query 78

A few queries get faster at larger scale factors. Queries 37 and 82 take about 0.3 s at SF1 and SF2 and less than 0.1 s from SF3 on. With the statistics of the larger tables, the optimizer probably chose a different plan for them. So the plan has a large effect on the query time, not only the data size.

5.6 Query 78

Query 78 compares sales quantities across the store, web and catalog channels. As shown in Table 7, its execution time varies considerably between scale factors and streams. At SF3, the Power Test execution took 1 127 seconds, compared with only 5.6 seconds at SF5. At SF7, it reached 13 099 seconds and accounted for approximately 97% of the total Power Test duration.

Such irregular behaviour suggests that changes in data volume alone do not explain the execution time. A change in the execution strategy is a plausible explanation. We therefore selected Query 78 for a more detailed execution plan analysis and SQL optimization experiment (section 7.1).

SF	Power (s)	TT1, 4 streams (s)	TT2, 4 streams (s)
1	0.7	1.5 – 2.5	1.4 – 1.5
2	1.5	3.8 – 5.1	2.4 – 3.9
3	1 127	2 611 – 2 681	5.5 – 8.1
5	5.6	12.8 – 27.2	6.3 – 23.7
7	13 099	13 172 – 13 428	35.8 (stream 8), 11 387 – 11 939

Table 7: Elapsed time of Query 78 in each test

At SF7, Query 78 alone is 97% of the Power Test time, and 90% at SF3.

5.7 Throughput Tests against the Power Test

In a Throughput Test, 4 streams run the 99 queries at the same time. Figure 7 shows the ratio T_{TT}/T_{Power}. A value of 1 would mean that the 4 streams together take as long as one stream, and a value of 4 that they actually run one after the other.

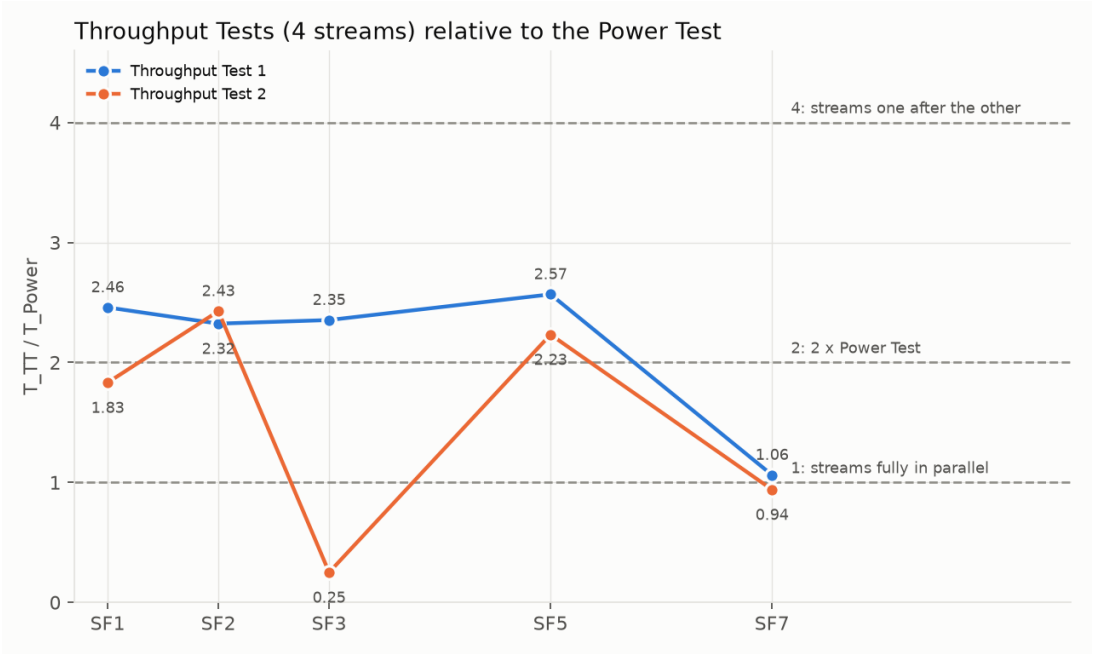


Figure 7: Elapsed time of the Throughput Tests relative to the Power Test

At SF1, SF2 and SF5, the ratio is between 1.8 and 2.6. Oracle Free can use at most 2 CPU threads, and a Power Test query runs on one thread. If the CPU is the limit, 4 streams on 2 threads should take about twice as long as one stream. The measured values are close to 2, so the CPU is the main bottleneck. The remaining 15 to 30% comes from the streams sharing the buffer cache, the disk and the 512 MB of PGA. At SF1, TT2 is faster than TT1 (1.83 against 2.46), probably because the data is already in memory after TT1 and the first data maintenance.

The points at SF3 and SF7 do not measure concurrency. At SF3, TT2 has a ratio of 0.25 because Query 78 was slow in the Power Test (1 127 s) and fast in TT2 (8 s at most). At SF7, both ratios are close to 1. Query 78 takes about 13 000 s in the Power Test and in almost every stream of the Throughput Tests, so this one query sets the length of all three tests.

5.8 Data Maintenance

Figure 8 shows the total time of the two data maintenance tests, split by phase.

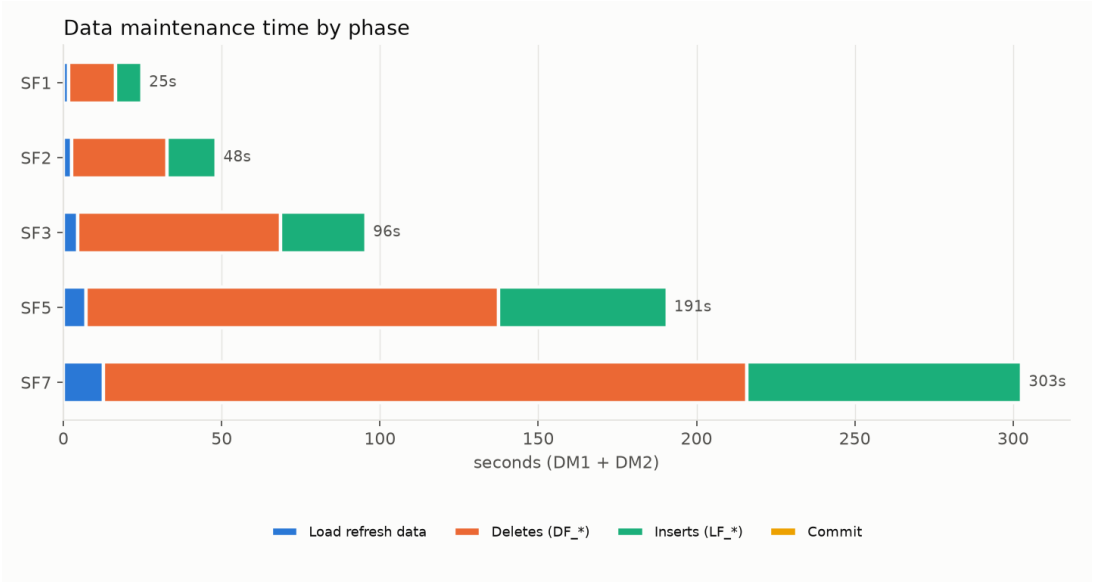


Figure 8: Time of the two data maintenance tests by phase

The data maintenance time grows from 25 s at SF1 to 303 s at SF7, so it is 12 times longer for 7 times more data. The deletes are always the largest phase, between 55% and 75% of each refresh run. To remove the sales and returns of a date range, Oracle has to find them in the fact tables. There is no index on the date columns, so each delete scans a large part of the table. Its cost therefore grows with the size of the fact table and not with the number of deleted rows. The inserts come second, and about 91% of the inserted rows are inventory rows. Loading the refresh data is short, and the commit is too short to be visible in the figure.

Per scale factor unit, the data maintenance takes about 25 s at SF1 and SF2, then 32 s at SF3, 38 s at SF5 and 43 s at SF7. This is the same buffer cache effect as for the queries: from SF3 on, the deletes have to read the fact tables from disk.

At SF7, DM1 took considerably longer than DM2. Additional disk activity and undo tablespace growth may have contributed to this difference, but their individual impact was not measured. The data maintenance stays small compared with the query tests. At SF5, the two runs take 191 s together, against 1 651 s for the two Throughput Tests.

5.9 QphDS across Scale Factors

Figure 9 shows the QphDS@SF metric at each scale factor.

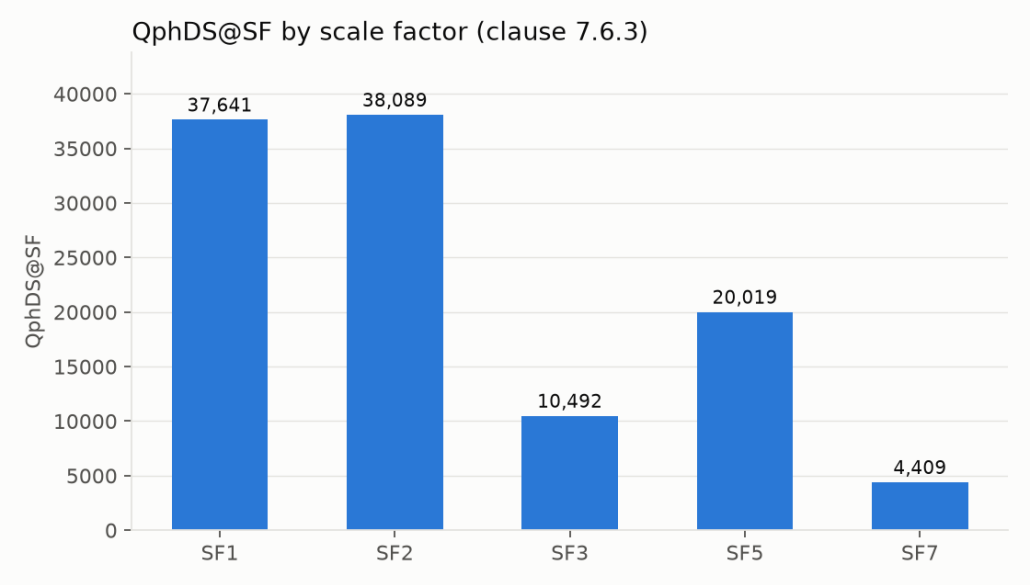


Figure 9: QphDS@SF by scale factor

We also calculated an adapted QphDS-style performance indicator using the formula defined in Clause 7.6.3 of the TPC-DS specification. It combines the load, power, throughput and data maintenance times.

Since our experiments use small, non-standard scale factors and an adapted benchmark configuration, these values are not official TPC-DS performance results. We use them only to summarize the behaviour of our experimental workload at each dataset size.However, we can note here as well, that the results are strongly affected by the irregular execution times of Query 78, especially at SF3 and SF7.

5.10 Limitations

- SF7 is at the limit of Oracle Free. During the run, the data maintenance and the large queries filled the undo tablespace (12.4 GB) and the temporary tablespace (10.9 GB), and the user data file grew to 12.9 GB. After a restart, Oracle refused to open the pluggable database (`ORA-12954`: the request exceeds the maximum allowed database size of 12 GB). The benchmark itself had finished and all the results were saved.
- With only 2 CPU threads and 906 MB of buffer cache, the results mostly show the limits of the free edition, not what Oracle Database can do on a larger machine.

6 Query Optimization

After completing the TPC-DS benchmark, we will focus on reducing query execution times and improving overall database performance.

6.1 Overview

Before rewriting individual queries, we applied a sequence of global changes to the database and measured each one against the TPC-DS workload. Every step was evaluated on the same set of queries, so its effect on the total runtime and on individual queries can be compared directly. The steps were:

1. Run the workload without optimizer statistics (baseline).
2. Gather optimizer statistics.
3. Declare the primary and foreign key relationships of the TPC-DS schema.
4. Add secondary indexes on the largest fact tables.
5. Create materialized views for repeated aggregation patterns.

Step	Effect on the workload
Gather statistics	Large improvement
Primary/foreign keys	No tangible improvement
Indexes	No tangible improvement execution plans unchanged
Materialized views	Targeted wins, but overall time increased

Table 8: Summary of the global optimization steps

6.2 Gathering statistics

Gathering statistics gave the largest improvement of all global changes. As detailed in Section 2.2, most of the overall execution time reduction comes from a small set of highly complex queries. Without statistics, the optimizer relies on default assumptions and cannot estimate cardinalities accurately, leading to suboptimal plan choices, such as selecting nested loops over large inputs instead of hash joins with full scans.

The impact of missing statistics becomes significantly more severe as the dataset grows:

- **Overall Runtime Reduction:** At SF1, collecting statistics cut the total benchmark execution time roughly in half (achieving an overall $\sim 2\times$ speedup). At SF5, the total execution time dropped by more than two-thirds, yielding an overall speedup exceeding $3\times$.
- **Compounding Performance Degradedness:** While queries running on smaller scale factors can often withstand bad join orders due to low baseline data volumes, scaling up the data exposes the exponential cost of poor cardinality estimation. At SF5, the lack of statistics caused severe regressions in outlier queries (with individual query speedups exceeding $100\times$), demonstrating that accurate statistics are critical for maintaining query scalability.

6.3 Primary and foreign key relationships

We then ran the script that defines all TPC-DS primary and foreign keys. This produced no tangible improvement. Constraints can help the optimizer in two ways: by eliminating joins that do not contribute columns to the result, and by enabling query rewrite. TPC-DS queries use columns from the dimensions they join, so there are few joins to eliminate, and the cardinality estimates were already provided by the statistics.

6.4 Optimizer configuration

We tested a few session parameters that control how the optimizer estimates cardinalities and which transformations it may apply. The defaults and the tested values are listed below.

6.4.1 Why we tested these changes.

Gathering statistics gives the optimizer table-level and column-level information, but not the correlation between columns or the effect of complex predicates, which TPC-DS queries use heavily. Raising `optimizer_dynamic_sampling` to 4 and 6 tests whether sampling at parse time corrects those estimates, at the cost of a longer parse. We tested `star_transformation_enabled` because TPC-DS is a star/snowflake schema, which is the case this transformation was designed for.

6.5 Indexes

We tested three indexes on the tables involved in the slowest queries:

```
CREATE INDEX idx_inv_date_item
ON inventory (inv_date_sk, inv_item_sk);

CREATE INDEX idx_cs_sold_date
ON catalog_sales (cs_sold_date_sk);

CREATE INDEX idx_cs_demo_item
ON catalog_sales (cs_bill_cdemo_sk, cs_item_sk);
```

Listing 1: Indexes Tested

6.5.0.1 Why we expected an improvement.

`inventory` and `catalog_sales` are large fact tables, and queries such as Query 72 join them by date, item and customer demographics. An index on the join and filter columns could replace a full scan with an index range scan and allow nested loop joins driven by a small set of dimension rows.

6.5.0.2 Why there was no improvement.

The execution times did not change and the execution plans stayed the same, which means the optimizer never chose the new indexes. The likely reason is that these queries are analytic and read a large fraction of each fact table. For such access patterns, a full scan with multiblock reads and a hash join is cheaper than a very large number of single-block index lookups, so the cost-based optimizer keeps the scan. At the Scale Fators Analyzed the tables are also small enough that full scans are fast.

6.6 Materialized views

6.6.1 How the views were designed

We looked for aggregation patterns that repeat across queries: customer, year and channel summaries in Q4, Q11 and Q74, and item, month and channel summaries in Q56 and Q60. For each pattern we created a materialized view at the shared grain and left the query-specific filters (year, category, GMT offset) out of the definition, so that Oracle can apply them on top of the stored aggregate.

6.6.2 Summary

The experiment added six Oracle materialized views containing reusable aggregates from the TPC-DS qualification workload. The views were designed to cover repeated customer/year/channel summaries in Q4, Q11, and Q74, and item/month/channel summaries in Q56 and Q60. They are defined in `create_materialized_views.sql`.

Table 9 lists the queries that improved at both scale factors, expressed as the speedup relative to the statistics-only baseline. The customer/year queries (Q4, Q11 and Q74) show the largest and most consistent gains, and Q60 also improves at both scales, although by less at SF5. Q56, which was a target, improved at SF1 but regressed at SF5. The reported execution plans (Figures 18,17,19) show materialized-view use for the queries that were inspected. At the workload level, the views give at most a marginal gain at SF1 and a clear slowdown at SF5, where most queries regress. The results therefore show targeted wins, but do not show a consistent overall runtime improvement.

Query	SF1 speedup	SF5 speedup
Q4	7.71×	5.09×
Q11	8.37×	7.83×
Q60	2.86×	1.71×
Q74	2.68×	2.90×
Q56	1.62x×	0.74x×

Table 9: Speedup of the materialized-view configuration over the baseline, for the queries that improved at both SF1 and SF5.

6.6.3 Views created and intended reuse

All six views use `BUILD IMMEDIATE`, `REFRESH COMPLETE ON DEMAND`, and `ENABLE QUERY REWRITE`. They do not filter to a particular qualification year, customer trait, product category, or GMT offset.

Views	Grouping grain and measures	Intended consumers
`MV_CUST_YEAR_STORE`, `MV_CUST_YEAR_CATALOG`, `MV_CUST_YEAR_WEB`	Customer attributes and year, with the measures used by these queries: Q4's calculated amount, Q11's list price less discount, and net paid for Q74.	Q4 uses all three sales channels; Q11 uses store and web; Q74 uses store and web. Separate views preserve the natural grain of each fact table.
`MV_ITEM_MONTH_STORE`, `MV_ITEM_MONTH_CATALOG`, `MV_ITEM_MONTH_WEB`	Item identifiers and category/color attributes, year, month, and billing-address GMT offset; summed extended sales price.	Q56 and Q60 sum item sales across channels and filter on item attributes, year/month, and GMT offset.

Table 10: Materialized views and the queries they target.

These are pre-aggregates rather than stored answers for a single qualification parameter set. Their extra item attributes allow item-category or item-color filters to be applied without hard-coding those filters into a view.

6.6.4 Why the total may still be worse

1. **Only a subset of queries benefit.** The views target Q4, Q11, Q74, Q56, and Q60. Most of the workload has different access patterns and cannot be expected to gain from these summaries.

2. **Runtime measurements vary.** Cache state, concurrent database activity, CPU and memory pressure, and run order can affect query times. A single SF1 comparison is evidence of a possible regression, not by itself proof of its cause.

3. **The reported total is a workload trade-off.** Large speedups on a few queries can be outweighed by small slowdowns across many others. The supplied total reflects that trade-off.

6.7 Limitations

All experiments were run at scale factors up to 7, which is the largest size we could execute considering the limitaions of Oracle free version. Our conclusions therefore apply to datasets of this size only. In particular, indexes and materialized views tend to become more useful as

tables grow, because scanning large fact tables becomes more expensive relative to an index lookup or a pre-aggregated view.

7 Query-level Optimization

After the global changes, we examined the queries individually and tried to reduce their runtime by changing the SQL syntax and restructuring the queries.

7.1 Query 78

From a business perspective, Query 78 compares customer purchases across different sales channels for the year 2000. It calculates the ratio of store sales quantities to the combined web and catalog sales quantities for the same item and customer, excluding returned sales. A 2 ratio means that the customer purchased twice as many units in the stores as through the other two combined channels.

```
WITH ws AS (
    SELECT
        d_year AS ws_sold_year,
        ws_item_sk,
        ws_bill_customer_sk ws_customer_sk,
        SUM(ws_quantity) ws_qty,
        SUM(ws_wholesale_cost) ws_wc,
        SUM(ws_sales_price) ws_sp
    FROM web_sales
    LEFT JOIN web_returns
        ON wr_order_number = ws_order_number
        AND ws_item_sk = wr_item_sk
    JOIN date_dim
        ON ws_sold_date_sk = d_date_sk
    WHERE wr_order_number IS NULL
    GROUP BY
        d_year, ws_item_sk, ws_bill_customer_sk
),

cs AS (
    SELECT
        d_year AS cs_sold_year,
        cs_item_sk,
        cs_bill_customer_sk cs_customer_sk,
        SUM(cs_quantity) cs_qty,
        SUM(cs_wholesale_cost) cs_wc,
        SUM(cs_sales_price) cs_sp
    FROM catalog_sales
    LEFT JOIN catalog_returns
        ON cr_order_number = cs_order_number
        AND cs_item_sk = cr_item_sk
    JOIN date_dim
```

```
        ON cs_sold_date_sk = d_date_sk
    WHERE cr_order_number IS NULL
    GROUP BY
        d_year, cs_item_sk, cs_bill_customer_sk
),

ss AS (
    SELECT
        d_year AS ss_sold_year,
        ss_item_sk,
        ss_customer_sk,
        SUM(ss_quantity) ss_qty,
        SUM(ss_wholesale_cost) ss_wc,
        SUM(ss_sales_price) ss_sp
    FROM store_sales
    LEFT JOIN store_returns
        ON sr_ticket_number = ss_ticket_number
        AND ss_item_sk = sr_item_sk
    JOIN date_dim
        ON ss_sold_date_sk = d_date_sk
    WHERE sr_ticket_number IS NULL
    GROUP BY
        d_year, ss_item_sk, ss_customer_sk
)

SELECT *
FROM (
    SELECT
        ss_sold_year,
        ss_item_sk,
        ss_customer_sk,

        ROUND(
            ss_qty / (
                COALESCE(ws_qty, 0) +
                COALESCE(cs_qty, 0)
            ), 2
        ) ratio,

        ss_qty store_qty,
        ss_wc store_wholesale_cost,
        ss_sp store_sales_price,

        COALESCE(ws_qty, 0) +
        COALESCE(cs_qty, 0) other_chan_qty,

        COALESCE(ws_wc, 0) +
        COALESCE(cs_wc, 0) other_chan_wholesale_cost,

        COALESCE(ws_sp, 0) +
        COALESCE(cs_sp, 0) other_chan_sales_price
```

```
84
85      FROM ss
86
87      LEFT JOIN ws
88          ON ws_sold_year = ss_sold_year
89          AND ws_item_sk = ss_item_sk
90          AND ws_customer_sk = ss_customer_sk
91
92      LEFT JOIN cs
93          ON cs_sold_year = ss_sold_year
94          AND cs_item_sk = ss_item_sk
95          AND cs_customer_sk = ss_customer_sk
96
97      WHERE (
98          COALESCE(ws_qty, 0) > 0
99          OR COALESCE(cs_qty, 0) > 0
100     )
101     AND ss_sold_year = 2000
102
103     ORDER BY
104         ss_sold_year,
105         ss_item_sk,
106         ss_customer_sk,
107         ss_qty DESC,
108         ss_wc DESC,
109         ss_sp DESC,
110         other_chan_qty,
111         other_chan_wholesale_cost,
112         other_chan_sales_price,
113         ratio
114 )
115 WHERE ROWNUM <= 100;
```

Listing 2: Original SQL Query 78

7.1.1 Query Structure

1. **Web Sales (ws):** Starting from `web_sales`, the query performs a `LEFT JOIN` with `web_returns` and excludes matching returns. It then joins `date_dim` to obtain the sale year and groups the records by year, item and customer, calculating the total quantity, wholesale cost and sales price.

2. **Catalog Sales (cs):** The same process is applied to `catalog_sales`, using `catalog_returns` to remove returned sales and `date_dim` to obtain the year. The remaining records are aggregated by the same attributes.

3. **Store Sales (ss):** The query processes `store_sales` in a similar way, joining `store_returns` and `date_dim`, excluding returned sales and computing the same aggregations.

4. **Final Query:** Starting from the aggregated store sales, two `LEFT JOIN` operations combine `ss` with `ws` and `cs` using the year, item and customer keys. The query then keeps only records from 2000 with a positive web or catalog sales quantity. It calculates the sales ratio and the combined quantities, costs and prices of the other channels. Finally, the results are sorted and limited to the first 100 rows.

7.1.2 Execution Plan Analysis

Id	Operation	Name	Starts	E-Rows	A-Rows	A-Time	Buffers	Reads	Writes	OMem	1Mem	Used-Mem	Used-Tmp
0	SELECT STATEMENT		1		100	00:18:23.81	1006M	109M	5310				
*1	COUNT STOPKEY		1		100	00:18:23.81	1006M	109M	5310				
2	VIEW		1	244	100	00:18:23.81	1006M	109M	5310				
*3	SORT ORDER BY STOPKEY		1	244	100	00:18:23.81	1006M	109M	5310	13312	13312	12288 (0)	
*4	FILTER		1		483	00:18:23.81	1006M	109M	5310				
5	NESTED LOOPS OUTER		1	244	1472K	00:18:23.59	1006M	109M	5310				
6	NESTED LOOPS OUTER		1	5	1472K	00:06:06.85	339M	37M	5310				
7	VIEW		1	1	1472K	00:00:02.22	128K	128K	5310				
8	HASH GROUP BY		1	1	1472K	00:00:02.00	128K	128K	5310	175M	9470K	73M (1)	45M
*9	FILTER		1		1491K	00:00:00.81	128K	123K	0				
*10	HASH JOIN RIGHT OUTER		1	1470K	1657K	00:00:00.79	128K	123K	0	48M	9892K	53M (0)	
11	INDEX FAST FULL SCAN	SYS_C008753	1	862K	862K	00:00:00.06	3425	81	0				
*12	HASH JOIN		1	1644K	1657K	00:00:00.52	125K	122K	0	2278K	2278K	1700K (0)	
*13	TABLE ACCESS FULL	DATE_DIM	1	363	366	00:00:00.01	1382	0	0				
14	TABLE ACCESS FULL	STORE_SALES	1	8639K	8639K	00:00:00.45	123K	122K	0				
*15	VIEW PUSHED PREDICATE		1472K	1	164	00:06:04.27	339M	37M	0				
16	SORT GROUP BY		1472K	1	31M	00:06:02.98	339M	37M	0	6144	6144	6144 (0)	
17	NESTED LOOPS		1472K	53	31M	00:05:55.42	339M	37M	0				
18	NESTED LOOPS		1472K	53	105M	00:05:16.47	233M	37M	0				
*19	FILTER		1472K		105M	00:04:38.17	126M	37M	0				
*20	HASH JOIN OUTER		1472K	53	117M	00:04:33.11	126M	37M	0	1162K	1162K	1686K (0)	
21	TABLE ACCESS BY INDEX ROWID BATCHED	WEB_SALES	1472K	60	117M	00:03:46.47	122M	36M	0				
*22	INDEX RANGE SCAN	SYS_C008777	1472K	60	117M	00:00:16.21	4826K	1547K	0				
*23	INDEX RANGE SCAN	SYS_C008774	1472K	6	11M	00:00:06.25	4449K	541K	0				
*24	INDEX UNIQUE SCAN	SYS_C008720	105M	1	105M	00:00:27.08	106M	2606	0				
*25	TABLE ACCESS BY INDEX ROWID	DATE_DIM	105M	1	31M	00:00:30.51	105M	16190	0				
*26	VIEW PUSHED PREDICATE		1472K	1	319	00:12:16.41	667M	72M	0				
27	SORT GROUP BY		1472K	1	63M	00:12:14.17	667M	72M	0	11264	11264	10240 (0)	
28	NESTED LOOPS		1472K	107	63M	00:12:00.43	667M	72M	0				
29	NESTED LOOPS		1472K	108	210M	00:10:48.03	456M	72M	0				
*30	FILTER		1472K		211M	00:09:34.38	244M	72M	0				
*31	HASH JOIN OUTER		1472K	108	235M	00:09:24.49	244M	72M	0	1162K	1162K	1742K (0)	
32	TABLE ACCESS BY INDEX ROWID BATCHED	CATALOG_SALES	1472K	120	235M	00:08:14.05	240M	71M	0				
*33	INDEX RANGE SCAN	SYS_C008780	1472K	120	235M	00:00:24.05	5267K	2063K	0				
*34	INDEX RANGE SCAN	SYS_C008771	1472K	12	23M	00:00:08.46	4476K	800K	0				
*35	INDEX UNIQUE SCAN	SYS_C008720	211M	1	210M	00:00:51.73	212M	2757	0				
*36	TABLE ACCESS BY INDEX ROWID	DATE_DIM	210M	1	63M	00:00:56.27	210M	21488	0				

Figure 10: Original execution plan of Query 78 on the benchmark machine.

As shown in Figure 10, Query 78 took approximately 18 minutes and 24 seconds (SF3). The main bottleneck comes from the two `VIEW PUSHED PREDICATE` operations (Id 15 and 26), used with `NESTED LOOPS OUTER` joins. Oracle repeatedly evaluates the web and catalog sales views for each group produced by the store sales CTE. Since this CTE produces approximately 1.47 million groups, both views are started around 1.47 million times, resulting in a large number of repeated table accesses. The plan also shows a significant estimation error: the store sales aggregation (Id 8) was estimated to produce one group but actually produced approximately 1.47 million. This may have contributed to the inefficient join strategy. The date filtering does not appear to be the main bottleneck, as most of the execution time comes

from the repeated processing of the web and catalog sales views. Our optimization therefore aims to avoid these repeated operations.

7.1.3 Optimization Experiments

To address the repeated processing identified in the original execution plan, we tested two optimization approaches. Our goal was to improve execution time while keeping the same query results. Both versions apply the year filter `d_year = 2000` directly inside the three sales CTEs.

7.1.3.1 SQL Rewriting with UNION ALL

The first approach rewrites the final part of the query using `UNION ALL` instead of two `LEFT JOIN` operations. The three sales channels are combined vertically, then grouped by year, item and customer. Conditional aggregations reconstruct the quantities and other measures for each channel, while preserving the original filters. The simplified structure is shown below.

```
WITH all_sales AS (
    SELECT ..., 'S' AS channel FROM ss
    UNION ALL
    SELECT ..., 'W' AS channel FROM ws
    UNION ALL
    SELECT ..., 'C' AS channel FROM cs
),
combined AS (
    SELECT year, item, customer,
        MAX(CASE WHEN channel = 'S'
            THEN qty END) AS store_qty,
        MAX(CASE WHEN channel = 'W'
            THEN qty END) AS web_qty,
        MAX(CASE WHEN channel = 'C'
            THEN qty END) AS catalog_qty
    FROM all_sales
    GROUP BY year, item, customer
)
SELECT ... FROM combined;
```

Listing 3: Simplified structure of the UNION ALL rewrite

This rewrite removes the two final joins responsible for the repeated view evaluations. However, it introduces an additional global aggregation.

7.1.3.2 Optimization with Oracle Hints

The second approach directly targets the inefficient execution plan identified earlier. Instead of rewriting the joins, we kept the original query structure and added optimizer hints to prevent the repeated evaluation of the web and catalog sales views.

```
SELECT /*+
    NO_MERGE(ws)
    NO_MERGE(cs)
    NO_PUSH_PRED(ws)
    NO_PUSH_PRED(cs)
*/
    ...
FROM ss
LEFT JOIN ws ON (...)
LEFT JOIN cs ON (...);
```

Listing 4: Optimizer hints added to Query 78

The NO_MERGE hints keep the web and catalog views separate, while N _PUSH_PRED prevents Oracle from applying the join conditions inside these views. This avoids the specific strategy that caused Oracle to process the same views millions of times in the original execution plan.

7.1.4 Results and Discussion

After testing both optimization methods, we selected the hint-based approach because it generally provided better execution times.

We note that the times of the original query in this section come from separate executions from those of the benchmark runs, which explains why they differ. This variation confirms that the execution time of Query 78 depends mainly on the plan chosen by Oracle, which is not stable from one execution to the next.

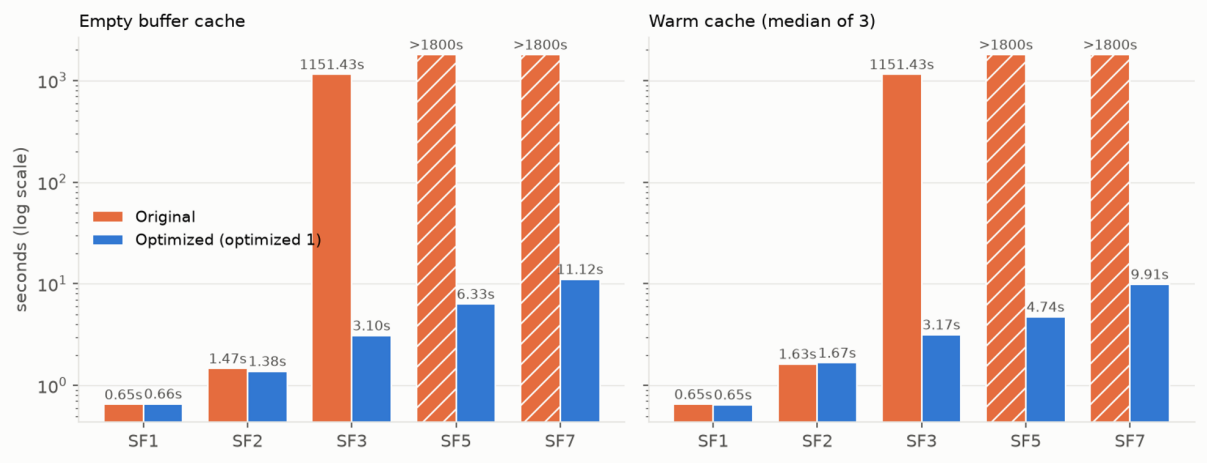


Figure 11: Query 78 execution times before and after optimization, with cold-cache and warm-cache measurements (logarithmic scale)

At SF1 and SF2, both versions had similar execution times because the original query was already fast. However, the improvement became significant for larger scale factors. At SF3, the original query took 1 151.43 seconds, compared with only 3.10 seconds using optimizer hints, making it approximately 371 times faster. At SF5 and SF7, the original query exceeded the 1 800 seconds, while the optimized version completed in 6.33 and 11.12 seconds, respectively.

The warm-cache measurements also show good results for the optimized version, with execution times of 3.17, 4.74 and 9.91 seconds at SF3, SF5 and SF7. The improvement can be explained by the new execution plan: Oracle uses `HASH JOIN OUTER` operations instead of repeatedly evaluating the web and catalog sales views (optimised plan fig 16). This avoids the main problem identified in the original plan.

Although the `UNION ALL` approach improved performance, but it required an additional `HASH GROUP BY` operation that processed millions rows at SF3 (execution plan fig 15). The hint-based approach was generally faster and kept the original SQL structure (so we retained this approach as our preferred optimization).

7.2 Query 72

7.2.1 Query Structure

As we have previously seen in section 5.4, Query 72 is one of the slowest queries. From a business perspective, for each item, warehouse and week, it counts the catalog sales with and without a promotion. Only sales. The query joins `catalog_sales` with `inventory`, the largest table in the workload, plus several dimensions making it very expensive.

```
SELECT * FROM (
    SELECT i_item_desc,
           w_warehouse_name,
           d1.d_week_seq,
           SUM(CASE WHEN p_promo_sk IS NULL
                    THEN 1 ELSE 0 END) no_promo,
           SUM(CASE WHEN p_promo_sk IS NOT NULL
                    THEN 1 ELSE 0 END) promo,
           COUNT(*) total_cnt
    FROM catalog_sales
    JOIN inventory ON (cs_item_sk = inv_item_sk)
    JOIN warehouse ON (w_warehouse_sk = inv_warehouse_sk)
    JOIN item ON (i_item_sk = cs_item_sk)
    JOIN customer_demographics ON (cs_bill_cdemo_sk = cd_demo_sk)
    JOIN household_demographics ON (cs_bill_hdemo_sk = hd_demo_sk)
    JOIN date_dim d1 ON (cs_sold_date_sk = d1.d_date_sk)
    JOIN date_dim d2 ON (inv_date_sk = d2.d_date_sk)
    JOIN date_dim d3 ON (cs_ship_date_sk = d3.d_date_sk)
    LEFT OUTER JOIN promotion ON (cs_promo_sk = p_promo_sk)
    LEFT OUTER JOIN catalog_returns
        ON (cr_item_sk = cs_item_sk
            AND cr_order_number = cs_order_number)
```

```
23      WHERE d1.d_week_seq = d2.d_week_seq
24        AND inv_quantity_on_hand < cs_quantity
25        AND d3.d_date > d1.d_date + 5
26        AND hd_buy_potential = '1001-5000'
27        AND d1.d_year = 2001
28        AND cd_marital_status = 'M'
29      GROUP BY i_item_desc, w_warehouse_name, d1.d_week_seq
30      ORDER BY total_cnt DESC, i_item_desc,
31               w_warehouse_name, d_week_seq
32  )
33  WHERE ROWNUM <= 100;
```

Listing 5: Original SQL Query 72

1. **Sales selection:** `catalog_sales` is joined with `date_dim` (`d1`, sale year 2001), `customer_demographics` (marital status) and `household_demographics` (buy potential). These three dimensions only filter the sales.
2. **Inventory match:** each sale is joined with `inventory` on the item only. The inventory date (`d2`) must fall in the same week as the sale (`d1.d_week_seq = d2.d_week_seq`) and the quantity on hand must be lower than the quantity sold.
3. **Shipping delay:** the ship date (`d3`) must be more than five days after the sale date.
4. **Left joins:** `promotion` is only used to tell whether a sale had a promotion, and no column of `catalog_returns` is used in the result.
5. **Aggregation:** the rows are grouped by item description, warehouse and week, sorted, and limited to the first 100 rows.

7.2.2 Reading the execution plan

Looking at the execution plan there are three things that we must focus on to understand what is making the query slow: the cumulative `A-Time` jumps (the time of a step is its `A-Time` minus that of its children), the gap between estimated (`E-Rows`) and actual (`A-Rows`) rows, and the temporary space used (`Writes` and `Used-Tmp`).

7.2.3 Execution Plan Analysis

Id	Operation	Name	Starts	E-Rows	A-Rows	A-Time	Buffers	Reads	Writes	OMem	1Mem	Used-Mem	Used-Tmp
0	SELECT STATEMENT		1		100	00:00:23.02	208K	481K	279K				
* 1	COUNT STOPKEY		1		100	00:00:23.02	208K	481K	279K				
2	VIEW		1	41890	100	00:00:23.02	208K	481K	279K				
* 3	SORT ORDER BY STOPKEY		1	41890	100	00:00:23.02	208K	481K	279K	80896	80896	71680 (0)	
4	HASH GROUP BY		1	41890	7643	00:00:23.02	208K	481K	279K	3021K	987K	3078K (0)	
5	NESTED LOOPS OUTER		1	41890	7760	00:00:23.02	208K	481K	279K				
* 6	HASH JOIN		1	41890	7760	00:00:23.02	208K	481K	279K	6587K	2318K	5733K (1)	6144K
7	TABLE ACCESS FULL	ITEM	1	36000	36000	00:00:00.01	2640	2639	0				
* 8	HASH JOIN		1	42030	7760	00:00:22.99	205K	478K	279K	5495K	2200K	4880K (0)	
9	TABLE ACCESS FULL	DATE_DIM	1	73049	73049	00:00:00.02	1382	1379	0				
* 10	HASH JOIN		1	844K	8093	00:00:22.96	204K	476K	279K	20M	6071K	7498K (1)	7168K
* 11	TABLE ACCESS FULL	CUSTOMER_DEMOGRAPHICS	1	384K	384K	00:00:00.08	15591	15588	0				
* 12	HASH JOIN		1	928K	40948	00:00:22.83	188K	460K	279K	2546K	2546K	1704K (0)	
* 13	TABLE ACCESS FULL	HOUSEHOLD_DEMOGRAPHICS	1	1200	1200	00:00:00.01	37	36	0				
* 14	HASH JOIN		1	5597K	245K	00:00:22.78	188K	460K	279K	3436M	44M	16M (1)	2241M
* 15	HASH JOIN		1	981K	39M	00:00:02.47	89933	87122	1320	1449K	1449K	1275K (0)	
16	TABLE ACCESS FULL	WAREHOUSE	1	6	6	00:00:00.01	6	6	0				
* 17	HASH JOIN		1	981K	39M	00:00:01.97	89927	87116	0	1316K	1316K	1462K (0)	
* 18	HASH JOIN		1	2544	2555	00:00:00.01	2764	0	0	1476K	1476K	1685K (0)	
* 19	TABLE ACCESS FULL	DATE_DIM	1	363	365	00:00:00.01	1382	0	0				
20	TABLE ACCESS FULL	DATE_DIM	1	73049	73049	00:00:00.01	1382	0	0				
21	TABLE ACCESS FULL	INVENTORY	1	28M	28M	00:00:01.00	87163	87116	0				
22	TABLE ACCESS FULL	CATALOG_SALES	1	4319K	4319K	00:00:00.77	95278	95273	0				
* 23	INDEX UNIQUE SCAN	SYS_C0010232	7760	1	7760	00:00:00.01	4	1	0				

Figure 12: Original execution plan of Query 72 on the benchmark machine.

At scale factor 3, the original query took 23 second (Figure 12). The plan has two problems, one leading to the other.

A large cardinality error. The join between the date pairs (`d1`, `d2`) and `inventory` (Id 17) is estimated at about 981 thousand rows, but produces 39 million, around 40 times more. The estimate is consistent with the optimizer spreading `inventory` evenly over all days of `date_dim` (about 73 thousand days), whereas `inventory` only holds one snapshot per week over a limited range of years, so each matching date has many more rows than the average. In addition, `d1` and `d2` are joined only on `d_week_seq`, so each inventory date matches several days of `d1`, which multiplies the rows again. The join of the two date tables itself (Id 18) is estimated correctly (2 544 estimated rows against 2 555 actual), so the error appears only when `inventory` is added.

A poor join order. Believing that the date–inventory side was small, the optimizer joined it with `catalog_sales` (4.3 million rows) first (Id 14). At that point `catalog_sales` had not yet been reduced by the filters on`customer_demographics` and `household_demographics`, which are only applied afterwards (Id 12 and 10). This join takes about 39 million plus 4.3 million rows as input and returns 245 thousand, so more than 99% of the work is discarded. An in-memory hash join would need about 3.4 GB (`OMem`), so it runs in one pass and spills about 2.2 GB to temporary disk (279 thousand blocks written, `Used-Tmp` 2 241 MB). This single step accounts for about 19.5 of the 23 seconds, roughly 85% of the elapsed time. The remaining joins (`d3`, `item`, `promotion`) only process about 8 thousand rows and cost almost nothing.

Oracle had also already removed the join with `catalog_returns` by itself (it does not appear in the plan), because it is joined through its primary key and none of its columns is used. The join with `promotion` is resolved with about 15 thousand index lookups.

7.2.4 Optimization Experiments

7.2.4.1 Moving conditions to the `ON` clauses.

For inner joins, `ON` and `WHERE` conditions are equivalent. Oracle merges them into one set of conditions and chooses the join order itself, so the plan did not change. The filters were already being applied as early as the chosen join order allowed (for instance `d_year = 2001` is applied when `d1` is scanned), so what needs to change is the order of the joins, not where the filters are written.

7.2.4.2 SQL rewriting with a CTE.

The second attempt restructured the query so that the filtered sales are computed first in a CTE and joined with `inventory` afterwards. At the same time, three joins were removed or simplified: the join with `catalog_returns` (unused), the join with `promotion`, replaced by the test `cs_promo_sk IS NULL`, and the join with `d3`, replaced by a comparison of the surrogate keys (`cs_ship_date_sk > cs_sold_date_sk + 5`), valid because `d_date_sk` increases by one per calendar day.

```sql
WITH s AS (
    SELECT cs_item_sk, cs_quantity, cs_promo_sk, d1.d_week_seq
    FROM catalog_sales
    JOIN date_dim d1 ON cs_sold_date_sk = d1.d_date_sk
                    AND d1.d_year = 2001
    JOIN customer_demographics ON cs_bill_cdemo_sk = cd_demo_sk
                    AND cd_marital_status = 'M'
    JOIN household_demographics ON cs_bill_hdemo_sk = hd_demo_sk
                    AND hd_buy_potential = '1001-5000'
    WHERE cs_ship_date_sk > cs_sold_date_sk + 5  -- replaces d3
)
SELECT * FROM (
    SELECT i_item_desc, w_warehouse_name, s.d_week_seq,
           SUM(CASE WHEN cs_promo_sk IS NULL
                    THEN 1 ELSE 0 END) no_promo,
           SUM(CASE WHEN cs_promo_sk IS NOT NULL
                    THEN 1 ELSE 0 END) promo,
           COUNT(*) total_cnt
    FROM s
    JOIN date_dim d2 ON d2.d_week_seq = s.d_week_seq
    JOIN inventory ON inv_date_sk = d2.d_date_sk
                  AND inv_item_sk = s.cs_item_sk
    JOIN warehouse ON w_warehouse_sk = inv_warehouse_sk
    JOIN item ON i_item_sk = s.cs_item_sk
    WHERE inv_quantity_on_hand < cs_quantity
    GROUP BY i_item_desc, w_warehouse_name, s.d_week_seq
```

```
    ORDER BY total_cnt DESC, i_item_desc,
             w_warehouse_name, d_week_seq
)
WHERE ROWNUM <= 100;
```

Listing 6: SQL rewrite of Query 72 with a CTE

After testing the rewrite in Listing 6, the optimizer plan did not change in the aspect that matters. Oracle merges a CTE that is not materialized back into the main query, so it is free to reorder the joins again. It starts from `d1` and `d2`, joins `inventory`, and only then joins `catalog_sales`, followed by `household_demographics` and `customer_demographics`. The join with `catalog_sales` again does not fit in memory and spills to temporary disk. Writing the query with the filters first is therefore not enough: the order is chosen by the optimizer from its estimates, and those estimates are the problem.

7.2.4.3 Optimizer hints.

We tried `LEADING` and `USE_HASH` to force the filtered `catalog_sales` to be joined with `inventory` last. A first attempt with the hints written after the outer `SELECT` had no effect, since the tables named in the hints are not visible from the outer query block (the `ROWNUM` filter prevents Oracle from merging that inline view). Placing the hints in the inner `SELECT`, on the original query, made them effective.

```
SELECT /*+ gather_plan_statistics */ *
FROM (
    SELECT /*+
        LEADING(d1 catalog_sales customer_demographics
                household_demographics d2 inventory)
        USE_HASH(inventory)
    */
        i_item_desc, w_warehouse_name, d1.d_week_seq, ...
    FROM catalog_sales
    JOIN inventory ON (cs_item_sk = inv_item_sk)
    ... -- rest of the original query unchanged
)
WHERE ROWNUM <= 100;
```

Listing 7: Optimizer hints added to Query 72

Id	Operation	Name	Starts	E-Rows	Cost (%CPU)	A-Rows	A-Time	Buffers	Reads	OMem	1Mem	Used-Mem	Used-Tmp
0	SELECT STATEMENT		1		345K(100)	100	00:00:02.39	204K	197K				
* 1	COUNT STOPKEY		1			100	00:00:02.39	204K	197K				
2	VIEW		1	820K	345K (1)	100	00:00:02.39	204K	197K				
* 3	SORT ORDER BY STOPKEY		1	820K	345K (1)	100	00:00:02.39	204K	197K	76800	76800	67584 (0)	
4	HASH GROUP BY		1	820K	345K (1)	7643	00:00:02.39	204K	197K	3021K	987K	2822K (0)	
* 5	HASH JOIN RIGHT OUTER		1	820K	208K (1)	7760	00:00:01.85	204K	197K	2402K	2402K	1743K (0)	
6	INDEX FULL SCAN	SYS_C008701	1	344	1 (0)	344	00:00:00.01	1	0				
* 7	HASH JOIN		1	820K	208K (1)	7760	00:00:01.79	204K	197K	1423K	1423K	1270K (0)	
8	TABLE ACCESS FULL	WAREHOUSE	1	6	3 (0)	6	00:00:00.01	6	0				
* 9	HASH JOIN		1	820K	208K (1)	7760	00:00:01.79	204K	197K	6587K	2318K	6782K (0)	
10	TABLE ACCESS FULL	ITEM	1	36000	716 (0)	36000	00:00:00.02	2643	0				
* 11	HASH JOIN		1	822K	202K (1)	7760	00:00:01.75	202K	197K	5495K	2200K	4853K (0)	
12	TABLE ACCESS FULL	DATE_DIM	1	73049	375 (0)	73049	00:00:00.01	1382	0				
* 13	HASH JOIN		1	16M	103K (1)	8093	00:00:01.49	200K	197K	17M	2994K	19M (0)	
* 14	HASH JOIN		1	898K	34808 (1)	197K	00:00:00.56	113K	110K	5086K	2395K	6731K (0)	
15	TABLE ACCESS FULL	DATE_DIM	1	73049	375 (0)	73049	00:00:00.01	1382	0				
* 16	HASH JOIN		1	128K	33761 (1)	28157	00:00:00.55	112K	110K	2546K	2546K	1755K (0)	
* 17	TABLE ACCESS FULL	HOUSEHOLD_DEMOGRAPHICS	1	1200	11 (0)	1200	00:00:00.01	37	0				
* 18	HASH JOIN		1	774K	33749 (1)	170K	00:00:00.57	112K	110K	20M	6071K	16M (0)	
* 19	TABLE ACCESS FULL	CUSTOMER_DEMOGRAPHICS	1	384K	4203 (1)	384K	00:00:00.94	15455	15452				
* 20	HASH JOIN		1	850K	26404 (1)	855K	00:00:00.31	96546	95135	1476K	1476K	1685K (0)	
* 21	TABLE ACCESS FULL	DATE_DIM	1	363	376 (1)	365	00:00:00.01	1382	0				
22	TABLE ACCESS FULL	CATALOG_SALES	1	4319K	26021 (1)	4319K	00:00:00.79	95164	95135				
23	TABLE ACCESS FULL	INVENTORY	1	28M	23941 (1)	28M	00:00:00.87	87242	87230				

Figure 13: Execution plan of Query 72 (using hints)

7.2.5 Results and Discussion

The hinted version runs in 2.3 seconds instead of 23, about 10 times faster. The new plan (Figure 13) is better for the following reasons.

1. **The selective filters are applied first.** `catalog_sales` is joined with `d1`, and the filters on `customer_demographics` and `household_demographics` then reduce it further, before `inventory` is touched.

2. **`inventory` is joined last and as the probe side.** The inventory rows are streamed against a small hash table built from the filtered sales, and the join directly returns only the final matching rows. The very large intermediate result no longer exists.

3. **The multiplication by week now happens on a small input.** Joining `d2` by week still produces several dates per sale, but it is applied to the already filtered sales instead of to the whole weekly inventory.

4. **Almost no temporary space is needed.** Temporary writes and physical reads drop substantially. The logical reads stay similar because both plans scan the same large tables once, so the gain comes from avoiding the spill and not from reading less data.

Both plans return the same number of rows at the end (7 760 rows before the aggregation, 7 643 groups), which is consistent with an equivalent result. The plan is still not perfect: `inventory` and `catalog_sales` are fully scanned (about 1.0 and 0.8 seconds), and the hash join with `customer_demographics` still writes a small amount to temporary disk, but these costs are small compared with the removed spill. The main limitation of this solution is that hints freeze the join order: it is correct for these parameters, but the optimizer cannot adapt if the data or the filter selectivity change.

7.3 Query 63

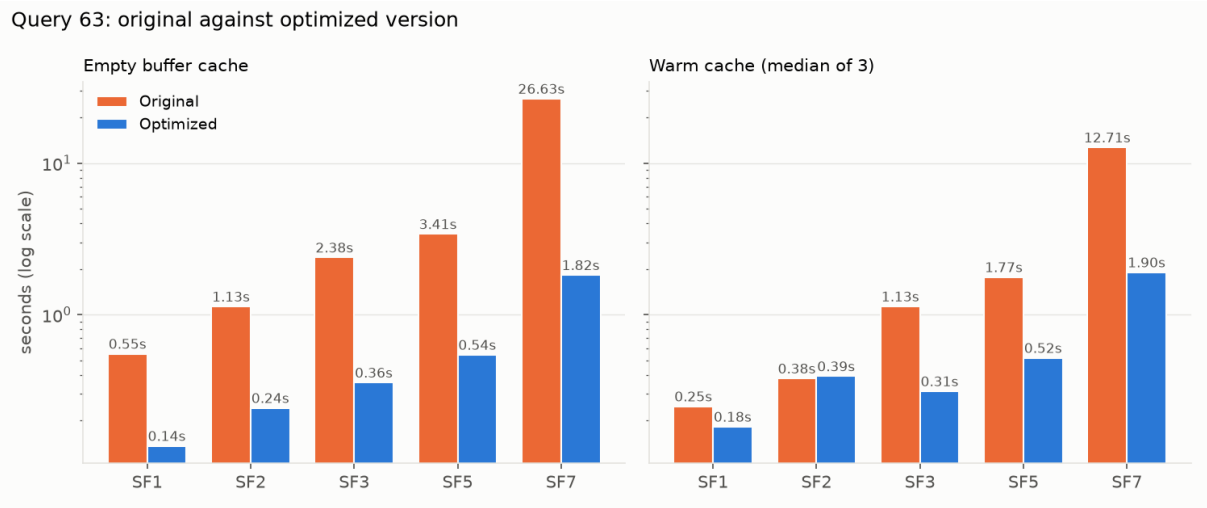


Figure 14: Query 63 execution times before and after optimization, with cold-cache and warm-cache measurements (logarithmic scale)

The optimized version is faster at every scale factor with an empty cache. The gap grows with the data size: at SF3 the time goes from 2.38 to 0.36 seconds (about 6.6 times faster), and at SF7 from 26.63 to 1.82 seconds (about 14.6 times faster). With a warm cache the gain is smaller at SF1 and SF2, where both versions already run in under 0.4 seconds. At SF7 it is still clear: 12.71 seconds against 1.90 seconds.

The main problem in the original plan was a wrong estimate on the `item` table. Oracle expected about 5 matching items, while about 1 900 actually match the filter. Because of this, it picked a plan suited to a small number of rows. In the optimized version, the `dynamic_sampling` hint makes Oracle count the real number of items, so it chooses a full scan with `HASH JOIN` operations. We also removed the join with `store`, since it was only there to drop sales without a store; a simple `ss_store_sk IS NOT NULL` condition does the same. The query now uses `WITH` clauses and computes the monthly average with a window function. The result is the same as with the original query at every scale factor.

8 Conclusion

This adapted implementation of TPC-DS on Oracle Database Free allowed us to evaluate database performance across five scale factors. We implemented the benchmark workflow, including data loading, Power and Throughput Tests, and Data Maintenance. The results showed that execution times generally increased with dataset size, although some queries were particularly slow or irregular, especially Query 78. By analyzing their execution plans and testing different optimization techniques, we identified the main bottlenecks and significantly improved the performance of the selected queries. These experiments also

highlighted the impact of Oracle Database Free's resource limitations on the benchmark results.

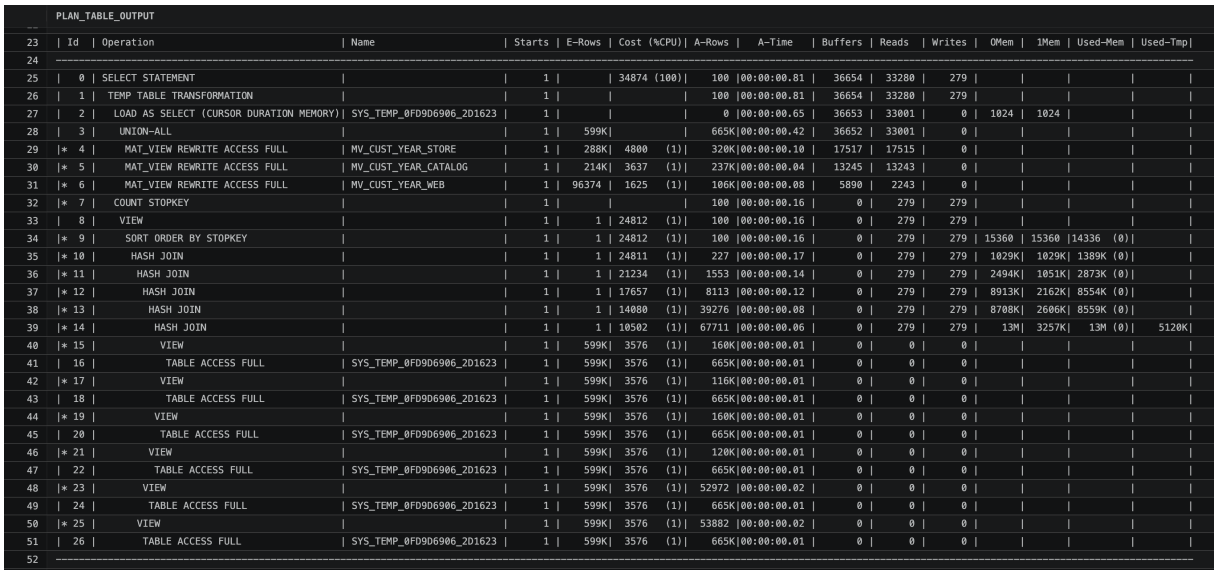
9 Appendix

Id	Operation	Name	Starts	E-Rows	A-Rows	A-Time	Buffers	Reads	Writes	OMem	1Mem	Used-Mem	Used-Tmp
0	SELECT STATEMENT		1		100	00:00:04.03	275K	306K	40389				
* 1	COUNT STOPKEY		1		100	00:00:04.03	275K	306K	40389				
2	VIEW		1	1	100	00:00:04.03	275K	306K	40389				
* 3	SORT ORDER BY STOPKEY		1	1	100	00:00:04.03	275K	306K	40389	15360	15360	14336 (0)	
* 4	FILTER		1		483	00:00:04.02	275K	306K	40389				
5	HASH GROUP BY		1	1	2617K	00:00:03.98	275K	306K	40389	708M	15M	75M (2)	235M
6	VIEW		1	1147	2617K	00:00:02.36	275K	275K	9420				
7	UNION-ALL		1	469	2617K	00:00:02.36	275K	275K	9420				
8	HASH GROUP BY		1	1	1456K	00:00:01.26	127K	128K	5535	179M	9374K	75M (1)	47M
* 9	FILTER		1		1456K	00:00:00.73	127K	122K	0				
* 10	HASH JOIN RIGHT OUTER		1	1419K	1618K	00:00:00.71	127K	122K	0	48M	9892K	53M (0)	
11	INDEX FAST FULL SCAN	SYS_C008753	1	862K	862K	00:00:00.03	3306	0	0				
* 12	HASH JOIN		1	1581K	1618K	00:00:00.50	124K	122K	0	1922K	1922K	1703K (0)	
* 13	TABLE ACCESS FULL	DATE_DIM	1	366	366	00:00:00.01	1382	0	0				
* 14	TABLE ACCESS FULL	STORE_SALES	1	8250K	8250K	00:00:00.42	122K	122K	0				
15	HASH GROUP BY		1	41	390K	00:00:00.33	49674	48412	960	49M	4687K	35M (1)	8192K
* 16	FILTER		1		390K	00:00:00.23	49674	47452	0				
* 17	HASH JOIN OUTER		1	8497	433K	00:00:00.22	49674	47452	0	35M	6377K	40M (0)	
* 18	HASH JOIN		1	433K	433K	00:00:00.11	48844	47452	0	1922K	1922K	1700K (0)	
* 19	TABLE ACCESS FULL	DATE_DIM	1	366	366	00:00:00.01	1382	0	0				
* 20	TABLE ACCESS FULL	WEB_SALES	1	2159K	2159K	00:00:00.09	47462	47452	0				
21	INDEX FAST FULL SCAN	SYS_C008774	1	215K	215K	00:00:00.01	830	0	0				
22	HASH GROUP BY		1	1105	771K	00:00:00.78	98298	98191	2925	97M	9374K	40M (1)	25M
* 23	FILTER		1		771K	00:00:00.52	98298	95266	0				
* 24	HASH JOIN OUTER		1	8525	857K	00:00:00.49	98298	95266	0	66M	6377K	77M (0)	
* 25	HASH JOIN		1	852K	857K	00:00:00.27	96660	95266	0	1922K	1922K	1704K (0)	
* 26	TABLE ACCESS FULL	DATE_DIM	1	366	366	00:00:00.01	1382	0	0				
* 27	TABLE ACCESS FULL	CATALOG_SALES	1	4297K	4297K	00:00:00.23	95278	95266	0				
28	INDEX FAST FULL SCAN	SYS_C008771	1	432K	432K	00:00:00.02	1638	0	0				

Figure 15: Optimised execution plan of Query 78 (rewriting using UNION ALL) on the benchmark machine at scale 3

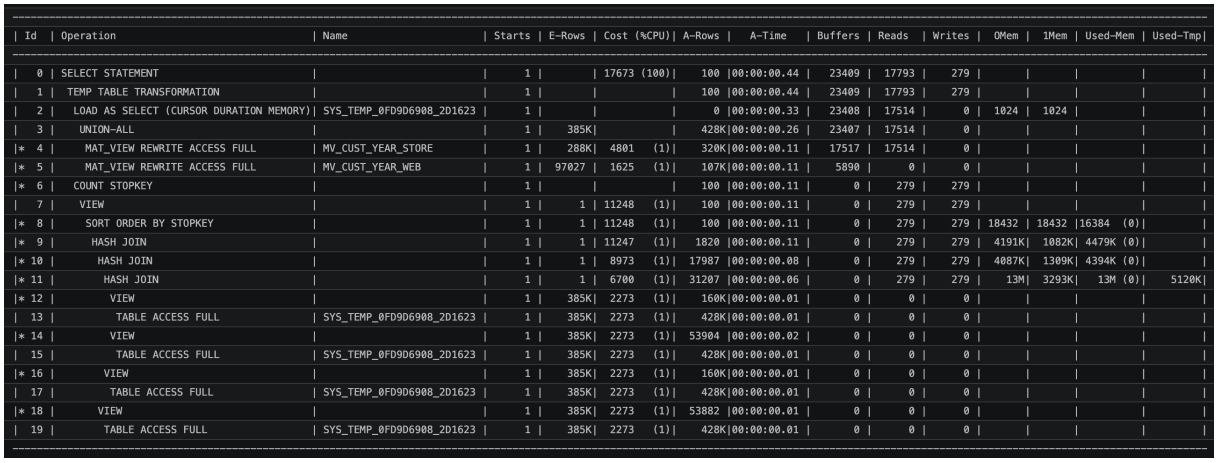
Id	Operation	Name	Starts	E-Rows	A-Rows	A-Time	Buffers	Reads	Writes	OMem	1Mem	Used-Mem	Used-Tmp
0	SELECT STATEMENT		1		100	00:00:03.13	275K	301K	35983				
* 1	COUNT STOPKEY		1		100	00:00:03.13	275K	301K	35983				
2	VIEW		1	1	100	00:00:03.13	275K	301K	35983				
* 3	SORT ORDER BY STOPKEY		1	1	100	00:00:03.13	275K	301K	35983	15360	15360	14336 (0)	
* 4	FILTER		1		483	00:00:03.11	275K	301K	35983				
* 5	HASH JOIN OUTER		1	1	1472K	00:00:03.09	275K	301K	35983	115M	12M	67M (1)	72M
* 6	HASH JOIN OUTER		1	1	1472K	00:00:02.64	225K	245K	26908	103M	6828K	54M (1)	72M
7	VIEW		1	1	1472K	00:00:01.85	127K	135K	12297				
8	HASH GROUP BY		1	1	1472K	00:00:01.85	127K	135K	12297	175M	9470K	67M (1)	42M
* 9	FILTER		1		1491K	00:00:01.21	127K	130K	7347				
* 10	HASH JOIN OUTER		1	16563	1657K	00:00:01.14	127K	130K	7347	122M	13M	80M (1)	64M
* 11	HASH JOIN		1	1656K	1657K	00:00:00.57	124K	122K	0	2278K	2278K	1685K (0)	
* 12	TABLE ACCESS FULL	DATE_DIM	1	366	366	00:00:00.01	1382	0	0				
13	TABLE ACCESS FULL	STORE_SALES	1	8639K	8639K	00:00:00.49	122K	122K	0				
14	INDEX FAST FULL SCAN	SYS_C008753	1	862K	862K	00:00:00.04	3306	0	0				
15	VIEW		1	1562	773K	00:00:00.58	98298	101K	6001				
16	HASH GROUP BY		1	1562	773K	00:00:00.58	98298	101K	6001	94M	9470K	49M (1)	19M
* 17	FILTER		1		773K	00:00:00.41	98298	99017	3751				
* 18	HASH JOIN OUTER		1	8568	859K	00:00:00.38	98298	99017	3751	63M	6662K	71M (0)	35M
* 19	HASH JOIN		1	856K	859K	00:00:00.20	96660	95266	0	2278K	2278K	1701K (0)	
* 20	TABLE ACCESS FULL	DATE_DIM	1	366	366	00:00:00.01	1382	0	0				
21	TABLE ACCESS FULL	CATALOG_SALES	1	4319K	4319K	00:00:00.17	95278	95266	0				
22	INDEX FAST FULL SCAN	SYS_C008771	1	432K	432K	00:00:00.02	1638	0	0				
23	VIEW		1	58	390K	00:00:00.26	49674	47917	465				
24	HASH GROUP BY		1	58	390K	00:00:00.26	49674	47917	465	47M	4735K	38M (1)	4096K
* 25	FILTER		1		390K	00:00:00.19	49674	47452	0				
* 26	HASH JOIN OUTER		1	8499	433K	00:00:00.18	49674	47452	0	33M	6662K	38M (0)	
* 27	HASH JOIN		1	433K	433K	00:00:00.10	48844	47452	0	2278K	2278K	1712K (0)	
* 28	TABLE ACCESS FULL	DATE_DIM	1	366	366	00:00:00.01	1382	0	0				
29	TABLE ACCESS FULL	WEB_SALES	1	2160K	2160K	00:00:00.08	47462	47452	0				
30	INDEX FAST FULL SCAN	SYS_C008774	1	215K	215K	00:00:00.01	830	0	0				

Figure 16: Optimised execution plan of Query 78 (using Oracle Hints) on the benchmark machine at scale 3



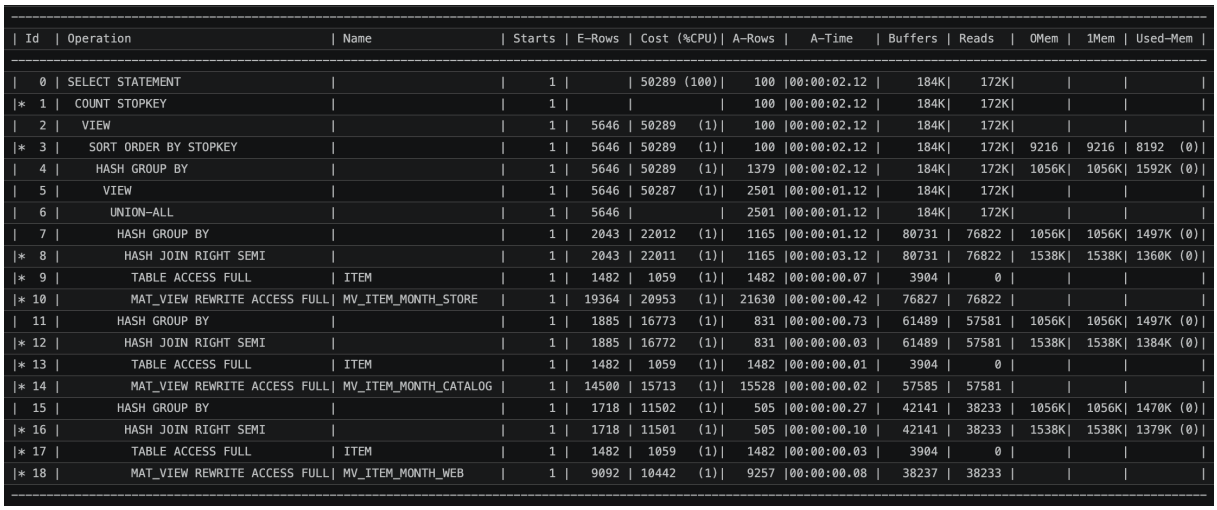
```
PLAN_TABLE_OUTPUT
| Id  | Operation                          | Name                     | Starts | E-Rows | Cost (%CPU)| A-Rows |   A-Time   | Buffers | Reads  | Writes |  OMem |  1Mem | Used-Mem | Used-Tmp|
|   0 | SELECT STATEMENT                   |                          |      1 |        | 34874 (100)|    100 |00:00:00.81 |   36654 |  33280 |    279 |       |       |          |         |
|   1 |  TEMP TABLE TRANSFORMATION         |                          |      1 |        |            |    100 |00:00:00.81 |   36654 |  33280 |    279 |       |       |          |         |
|   2 |   LOAD AS SELECT (CURSOR DURATION MEMORY)| SYS_TEMP_0FD9D6906_2D1623 |      1 |        |            |      0 |00:00:00.65 |   36653 |  33001 |      0 |  1024 |  1024 |          |         |
|   3 |    UNION-ALL                       |                          |      1 |   599K |            |   665K|00:00:00.42 |   36652 |  33001 |      0 |       |       |          |         |
|*  4 |     MAT_VIEW REWRITE ACCESS FULL   | MV_CUST_YEAR_STORE       |      1 |   288K|  4800   (1)|   320K|00:00:00.10 |   17517 |  17515 |      0 |       |       |          |         |
|*  5 |     MAT_VIEW REWRITE ACCESS FULL   | MV_CUST_YEAR_CATALOG     |      1 |   214K|  3637   (1)|   237K|00:00:00.04 |   13245 |  13243 |      0 |       |       |          |         |
|*  6 |     MAT_VIEW REWRITE ACCESS FULL   | MV_CUST_YEAR_WEB         |      1 |  96374 |  1625   (1)|   106K|00:00:00.08 |    5890 |   2243 |      0 |       |       |          |         |
|*  7 |    COUNT STOPKEY                   |                          |      1 |        |            |   100 |00:00:00.16 |       0 |    279 |    279 |       |       |          |         |
|   8 |     VIEW                           |                          |      1 |      1 | 24812   (1)|   100 |00:00:00.16 |       0 |    279 |    279 |       |       |          |         |
|*  9 |      SORT ORDER BY STOPKEY         |                          |      1 |      1 | 24812   (1)|   100 |00:00:00.16 |       0 |    279 |    279 | 15360 | 15360 |14336  (0)|         |
|* 10 |       HASH JOIN                    |                          |      1 |      1 | 24811   (1)|   227 |00:00:00.17 |       0 |    279 |    279 |  1029K|  1029K| 1389K (0)|         |
|* 11 |        HASH JOIN                   |                          |      1 |      1 | 21234   (1)|  1553 |00:00:00.14 |       0 |    279 |    279 |  2494K|  1051K| 2873K (0)|         |
|* 12 |         HASH JOIN                  |                          |      1 |      1 | 17657   (1)|  8113 |00:00:00.12 |       0 |    279 |    279 |  8913K|  2162K| 8554K (0)|         |
|* 13 |          HASH JOIN                 |                          |      1 |      1 | 14080   (1)| 39276 |00:00:00.08 |       0 |    279 |    279 |  8708K|  2606K| 8559K (0)|         |
|* 14 |           HASH JOIN                |                          |      1 |      1 | 10502   (1)| 67711 |00:00:00.06 |       0 |    279 |    279 |    13M|  3257K|    13M (0)|    5120K|
|* 15 |            VIEW                    |                          |      1 |   599K|  3576   (1)|   160K|00:00:00.01 |       0 |      0 |      0 |       |       |          |         |
|  16 |             TABLE ACCESS FULL      | SYS_TEMP_0FD9D6906_2D1623 |      1 |   599K|  3576   (1)|   665K|00:00:00.01 |       0 |      0 |      0 |       |       |          |         |
|* 17 |            VIEW                    |                          |      1 |   599K|  3576   (1)|   116K|00:00:00.01 |       0 |      0 |      0 |       |       |          |         |
|  18 |             TABLE ACCESS FULL      | SYS_TEMP_0FD9D6906_2D1623 |      1 |   599K|  3576   (1)|   665K|00:00:00.01 |       0 |      0 |      0 |       |       |          |         |
|* 19 |           VIEW                     |                          |      1 |   599K|  3576   (1)|   160K|00:00:00.01 |       0 |      0 |      0 |       |       |          |         |
|  20 |            TABLE ACCESS FULL       | SYS_TEMP_0FD9D6906_2D1623 |      1 |   599K|  3576   (1)|   665K|00:00:00.01 |       0 |      0 |      0 |       |       |          |         |
|* 21 |          VIEW                      |                          |      1 |   599K|  3576   (1)|   120K|00:00:00.01 |       0 |      0 |      0 |       |       |          |         |
|  22 |           TABLE ACCESS FULL        | SYS_TEMP_0FD9D6906_2D1623 |      1 |   599K|  3576   (1)|   665K|00:00:00.01 |       0 |      0 |      0 |       |       |          |         |
|* 23 |         VIEW                       |                          |      1 |   599K|  3576   (1)|  52972 |00:00:00.02 |       0 |      0 |      0 |       |       |          |         |
|  24 |          TABLE ACCESS FULL         | SYS_TEMP_0FD9D6906_2D1623 |      1 |   599K|  3576   (1)|   665K|00:00:00.01 |       0 |      0 |      0 |       |       |          |         |
|* 25 |        VIEW                        |                          |      1 |   599K|  3576   (1)|  53882 |00:00:00.02 |       0 |      0 |      0 |       |       |          |         |
|  26 |         TABLE ACCESS FULL          | SYS_TEMP_0FD9D6906_2D1623 |      1 |   599K|  3576   (1)|   665K|00:00:00.01 |       0 |      0 |      0 |       |       |          |         |
```

Figure 17: Execution plan of Query 4 (using materialized Views) on the benchmark machine at scale 5



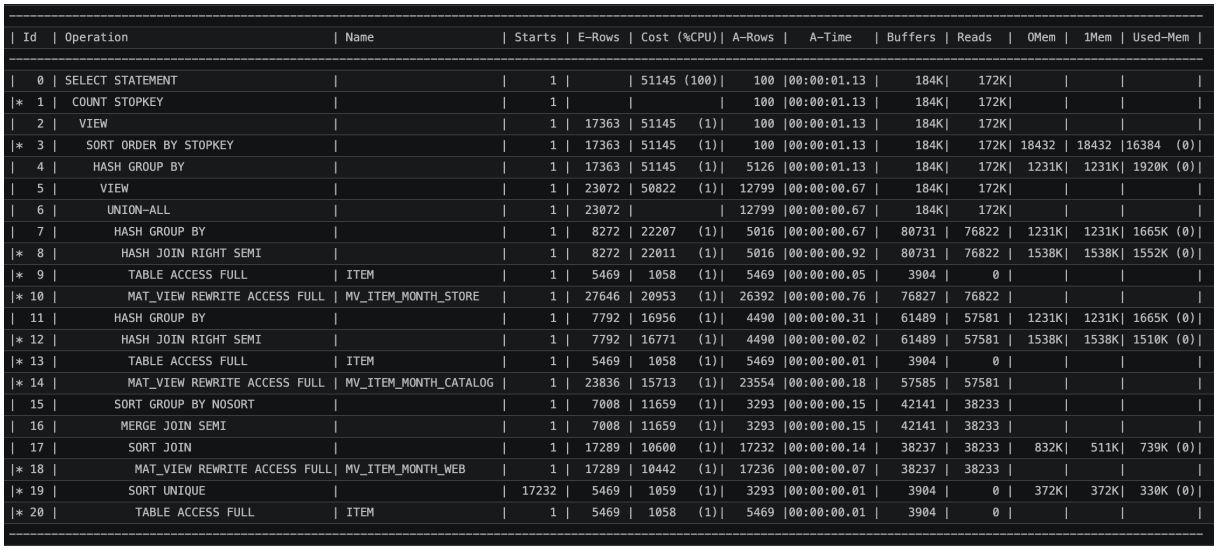
```
| Id  | Operation                          | Name                     | Starts | E-Rows | Cost (%CPU)| A-Rows |   A-Time   | Buffers | Reads  | Writes |  OMem |  1Mem | Used-Mem | Used-Tmp|
|   0 | SELECT STATEMENT                   |                          |      1 |        | 17673 (100)|    100 |00:00:00.44 |   23409 |  17793 |    279 |       |       |          |         |
|   1 |  TEMP TABLE TRANSFORMATION         |                          |      1 |        |            |    100 |00:00:00.44 |   23409 |  17793 |    279 |       |       |          |         |
|   2 |   LOAD AS SELECT (CURSOR DURATION MEMORY)| SYS_TEMP_0FD9D6908_2D1623 |      1 |        |            |      0 |00:00:00.33 |   23408 |  17514 |      0 |  1024 |  1024 |          |         |
|   3 |    UNION-ALL                       |                          |      1 |   385K|            |   428K|00:00:00.26 |   23407 |  17514 |      0 |       |       |          |         |
|*  4 |     MAT_VIEW REWRITE ACCESS FULL   | MV_CUST_YEAR_STORE       |      1 |   288K|  4801   (1)|   320K|00:00:00.11 |   17517 |  17514 |      0 |       |       |          |         |
|*  5 |     MAT_VIEW REWRITE ACCESS FULL   | MV_CUST_YEAR_WEB         |      1 |  97027 |  1625   (1)|   107K|00:00:00.11 |    5890 |      0 |      0 |       |       |          |         |
|*  6 |    COUNT STOPKEY                   |                          |      1 |        |            |   100 |00:00:00.11 |       0 |    279 |    279 |       |       |          |         |
|   7 |     VIEW                           |                          |      1 |      1 | 11248   (1)|   100 |00:00:00.11 |       0 |    279 |    279 |       |       |          |         |
|*  8 |      SORT ORDER BY STOPKEY         |                          |      1 |      1 | 11248   (1)|   100 |00:00:00.11 |       0 |    279 |    279 | 18432 | 18432 |16384  (0)|         |
|*  9 |       HASH JOIN                    |                          |      1 |      1 | 11247   (1)|  1820 |00:00:00.11 |       0 |    279 |    279 |  4191K|  1082K| 4479K (0)|         |
|* 10 |        HASH JOIN                   |                          |      1 |      1 |  8973   (1)| 17987 |00:00:00.08 |       0 |    279 |    279 |  4087K|  1309K| 4394K (0)|         |
|* 11 |         HASH JOIN                  |                          |      1 |      1 |  6700   (1)| 31207 |00:00:00.06 |       0 |    279 |    279 |    13M|  3293K|    13M (0)|    5120K|
|* 12 |          VIEW                      |                          |      1 |   385K|  2273   (1)|   160K|00:00:00.01 |       0 |      0 |      0 |       |       |          |         |
|  13 |           TABLE ACCESS FULL        | SYS_TEMP_0FD9D6908_2D1623 |      1 |   385K|  2273   (1)|   428K|00:00:00.01 |       0 |      0 |      0 |       |       |          |         |
|* 14 |          VIEW                      |                          |      1 |   385K|  2273   (1)|  53904 |00:00:00.02 |       0 |      0 |      0 |       |       |          |         |
|  15 |           TABLE ACCESS FULL        | SYS_TEMP_0FD9D6908_2D1623 |      1 |   385K|  2273   (1)|   428K|00:00:00.01 |       0 |      0 |      0 |       |       |          |         |
|* 16 |         VIEW                       |                          |      1 |   385K|  2273   (1)|   160K|00:00:00.01 |       0 |      0 |      0 |       |       |          |         |
|  17 |          TABLE ACCESS FULL         | SYS_TEMP_0FD9D6908_2D1623 |      1 |   385K|  2273   (1)|   428K|00:00:00.01 |       0 |      0 |      0 |       |       |          |         |
|* 18 |        VIEW                        |                          |      1 |   385K|  2273   (1)|  53882 |00:00:00.01 |       0 |      0 |      0 |       |       |          |         |
|  19 |         TABLE ACCESS FULL          | SYS_TEMP_0FD9D6908_2D1623 |      1 |   385K|  2273   (1)|   428K|00:00:00.01 |       0 |      0 |      0 |       |       |          |         |
```

Figure 18: Execution plan of Query 11 (using materialized Views) on the benchmark machine at scale 5



Id	Operation	Name	Starts	E-Rows	Cost (%CPU)	A-Rows	A-Time	Buffers	Reads	OMem	1Mem	Used-Mem
0	SELECT STATEMENT		1		50289 (100)	100	00:00:02.12	184K	172K			
* 1	COUNT STOPKEY		1			100	00:00:02.12	184K	172K			
2	VIEW		1	5646	50289 (1)	100	00:00:02.12	184K	172K			
* 3	SORT ORDER BY STOPKEY		1	5646	50289 (1)	100	00:00:02.12	184K	172K	9216	9216	8192 (0)
4	HASH GROUP BY		1	5646	50289 (1)	1379	00:00:02.12	184K	172K	1056K	1056K	1592K (0)
5	VIEW		1	5646	50287 (1)	2501	00:00:01.12	184K	172K			
6	UNION-ALL		1	5646		2501	00:00:01.12	184K	172K			
7	HASH GROUP BY		1	2043	22012 (1)	1165	00:00:01.12	80731	76822	1056K	1056K	1497K (0)
* 8	HASH JOIN RIGHT SEMI		1	2043	22011 (1)	1165	00:00:03.12	80731	76822	1538K	1538K	1360K (0)
* 9	TABLE ACCESS FULL	ITEM	1	1482	1059 (1)	1482	00:00:00.07	3904	0			
* 10	MAT_VIEW REWRITE ACCESS FULL	MV_ITEM_MONTH_STORE	1	19364	20953 (1)	21630	00:00:00.42	76827	76822			
11	HASH GROUP BY		1	1885	16773 (1)	831	00:00:00.73	61489	57581	1056K	1056K	1497K (0)
* 12	HASH JOIN RIGHT SEMI		1	1885	16772 (1)	831	00:00:00.03	61489	57581	1538K	1538K	1384K (0)
* 13	TABLE ACCESS FULL	ITEM	1	1482	1059 (1)	1482	00:00:00.01	3904	0			
* 14	MAT_VIEW REWRITE ACCESS FULL	MV_ITEM_MONTH_CATALOG	1	14500	15713 (1)	15528	00:00:00.02	57585	57581			
15	HASH GROUP BY		1	1718	11502 (1)	505	00:00:00.27	42141	38233	1056K	1056K	1470K (0)
* 16	HASH JOIN RIGHT SEMI		1	1718	11501 (1)	505	00:00:00.10	42141	38233	1538K	1538K	1379K (0)
* 17	TABLE ACCESS FULL	ITEM	1	1482	1059 (1)	1482	00:00:00.03	3904	0			
* 18	MAT_VIEW REWRITE ACCESS FULL	MV_ITEM_MONTH_WEB	1	9092	10442 (1)	9257	00:00:00.08	38237	38233			

Figure 19: Execution plan of Query 74 (using materialized Views) on the benchmark machine at scale 5



Id	Operation	Name	Starts	E-Rows	Cost (%CPU)	A-Rows	A-Time	Buffers	Reads	OMem	1Mem	Used-Mem
0	SELECT STATEMENT		1		51145 (100)	100	00:00:01.13	184K	172K			
* 1	COUNT STOPKEY		1			100	00:00:01.13	184K	172K			
2	VIEW		1	17363	51145 (1)	100	00:00:01.13	184K	172K			
* 3	SORT ORDER BY STOPKEY		1	17363	51145 (1)	100	00:00:01.13	184K	172K	18432	18432	16384 (0)
4	HASH GROUP BY		1	17363	51145 (1)	5126	00:00:01.13	184K	172K	1231K	1231K	1920K (0)
5	VIEW		1	23072	50822 (1)	12799	00:00:00.67	184K	172K			
6	UNION-ALL		1	23072		12799	00:00:00.67	184K	172K			
7	HASH GROUP BY		1	8272	22207 (1)	5016	00:00:00.67	80731	76822	1231K	1231K	1665K (0)
* 8	HASH JOIN RIGHT SEMI		1	8272	22011 (1)	5016	00:00:00.92	80731	76822	1538K	1538K	1552K (0)
* 9	TABLE ACCESS FULL	ITEM	1	5469	1058 (1)	5469	00:00:00.05	3904	0			
* 10	MAT_VIEW REWRITE ACCESS FULL	MV_ITEM_MONTH_STORE	1	27646	20953 (1)	26392	00:00:00.76	76827	76822			
11	HASH GROUP BY		1	7792	16956 (1)	4490	00:00:00.31	61489	57581	1231K	1231K	1665K (0)
* 12	HASH JOIN RIGHT SEMI		1	7792	16771 (1)	4490	00:00:00.02	61489	57581	1538K	1538K	1510K (0)
* 13	TABLE ACCESS FULL	ITEM	1	5469	1058 (1)	5469	00:00:00.01	3904	0			
* 14	MAT_VIEW REWRITE ACCESS FULL	MV_ITEM_MONTH_CATALOG	1	23836	15713 (1)	23554	00:00:00.18	57585	57581			
15	SORT GROUP BY NOSORT		1	7008	11659 (1)	3293	00:00:00.15	42141	38233			
16	MERGE JOIN SEMI		1	7008	11659 (1)	3293	00:00:00.15	42141	38233			
17	SORT JOIN		1	17289	10600 (1)	17232	00:00:00.14	38237	38233	832K	511K	739K (0)
* 18	MAT_VIEW REWRITE ACCESS FULL	MV_ITEM_MONTH_WEB	1	17289	10442 (1)	17236	00:00:00.07	38237	38233			
* 19	SORT UNIQUE		17232	5469	1059 (1)	3293	00:00:00.01	3904	0	372K	372K	330K (0)
* 20	TABLE ACCESS FULL	ITEM	1	5469	1058 (1)	5469	00:00:00.01	3904	0			

Figure 20: Execution plan of Query 60 (using materialized Views) on the benchmark machine at scale 5

References

[1] Transaction Processing Performance Council. *TPC Current Specifications and Benchmark Tools Downloads*. https://www.tpc.org/tpc_documents_current_versions/current_specifications5.asp

[2] Oracle Corporation. (2019). *Oracle Database SQL Tuning Guide, 19c: Gathering Optimizer Statistics*. https://docs.oracle.com/en/database/oracle/oracle-database/19/tgsql/gathering-optimizer-statistics.html

[3] Oracle Corporation. (2021). *Oracle Database Database Performance Tuning Guide, 21c*. https://docs.oracle.com/en/database/oracle/oracle-database/21/tgsql/

[4] Oracle Corporation. (2019). *Oracle Database Database PL/SQL Packages and Types Reference, 19c: DBMS_XPLAN*. https://docs.oracle.com/en/database/oracle/oracle-database/19/arpls/DBMS_XPLAN.html

[5] Oracle Corporation. (2021). *Oracle Database SQL Language Reference, 21c: EXPLAIN PLAN*. https://docs.oracle.com/en/database/oracle/oracle-database/21/sqlrf/EXPLAIN-PLAN.html

[6] Oracle Optimizer Team. (2012, March 9). *How to generate a useful SQL execution plan*. Oracle Optimizer Blog. https://blogs.oracle.com/optimizer/post/how-to-generate-a-useful-sql-execution-plan

The scripts used to run the benchmark were generated with the assistance of an AI tool (Claude, Anthropic [**anthropic_claude**]). They were reviewed, executed and validated by the authors.